Voice of Village: Fund Transparency & Issue Reporting System

Major project submitted in partial fulfillment of the requirement for the award of the degree of

BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE AND ENGINEERING

Under the esteemed guidance of

CH. Sandhya
Assistant Professor

By

V. Satwik Reddy - 22R11A05N9

Department of Computer Science and Engineering

Geethanjali College of Engineering and Technology (Autonomous)

Accredited by NAAC with A+ Grade: B.Tech. CSE, EEE, ECE accredited by NBA Sy. No: 33 & 34, Cheeryal (V), Keesara (M), Medchal District, Telangana – 501301

APRIL – 2026

Geethanjali College of Engineering and Technology (Autonomous)

Accredited by NAAC with A^{+} Grade : B.Tech. CSE, EEE, ECE accredited by NBA Sy. No: 33 & 34, Cheeryal (V), Keesara (M), Medchal District, Telangana – 501301

Department of Computer Science and Engineering

CERTIFICATE

This is to certify that the B.Tech Major Project report entitled **"Voice of Village: Fund Transparency & Issue Reporting System"** is a bonafide work done by Name of the student **V.Satwik Reddy - 22R11A05N9** in partial fulfillment of the requirement of the award for the degree of Bachelor of Technology in "**Computer Science and Engineering**" from Jawaharlal Nehru Technological University, Hyderabad during the year 2025-2026.

HoD – CSE

CH. Sandhya
Assistant Professor

Dr.E. Ravindra
Professor

External Examiner

Geethanjali College of Engineering and Technology (Autonomous)

Accredited by NAAC with A+ Grade : B.Tech. CSE, EEE, ECE accredited by NBA Sy. No: 33 & 34, Cheeryal (V), Keesara (M), Medchal District, Telangana – 501301

Department of Computer Science and Engineering

DECLARATION BY THE CANDIDATE

I **V. Satwik Reddy** bearing Roll No. **22R11A05N9** hereby declare that the project seminar report entitled **"Voice of Village: Fund Transparency & Issue Reporting System"** is done under the guidance of **Ms.Ch.Sandhya**, **Assistant Professor**, Department of Computer Science and Engineering, Geethanjali College of Engineering and Technology, is submitted in partial fulfillment of the requirements for the award of the degree of **Bachelor of Technology in Computer Science and Engineering**.

This is a record of bonafide work carried out by me and the results embodied in this project have not been reproduced or copied from any source. The results embodied in this project report have not been submitted to any other University or Institute for the award of any other degree or diploma.

V. Satwik Reddy "22R11A05N9"

Department of CSE, Geethanjali College of Engineering and Technology, Cheeryal

ACKNOWLEDGEMENT

It is with profound gratitude and sincere appreciation that I extend my heartfelt thanks to all those who have played a significant role in the successful completion of this undergraduate project.

First and foremost, I express my deep sense of respect and gratitude to our **Honourable Chairman, Mr. G. R. Ravinder Reddy**, for his constant encouragement and for nurturing a culture of academic excellence and innovation within the institution.

I would also like to express my sincere thanks to **Dr. S. Udaya Kumar**, **Director,** for his visionary leadership and continued support, which have provided the ideal environment and motivation for academic pursuits such as this project.

My heartfelt appreciation goes to **Dr. K. Sagar**, **Principal,** for his steadfast guidance, infrastructural support, and encouragement that have helped bring this project to successful fruition.

I am deeply grateful to **Dr. E. Ravindra, Professor**, **Head of the Department-CSE**, for his academic leadership, valuable feedback, and continuous support throughout the duration of this project.

I extend my sincere thanks to our **Project Coordinators, Mr. K. Kamakshaiah, Ms. K. Ashwini**, for their meticulous planning, timely evaluations, and constant coordination that ensured a smooth and structured project development process.

A special note of gratitude is reserved for our project guide, **Ms. CH. Sandhya, Assistant Professor**, whose expert supervision, insightful suggestions, and dedicated mentorship have been instrumental in shaping the direction and outcome of this work.

Lastly, I am ever grateful to our **parents and family** for their unconditional love, encouragement, and moral support. Their faith in us has been our greatest strength throughout this journey.

With genuine appreciation, I acknowledge every individual who has, in one way or another, contributed to the successful completion of this project.

With warm regards,

V. Satwik Reddy

22R11A05N9

Department of Computer Science and Engineering Geethanjali College of Engineering and Technology

ABSTRACT

In recent years, ensuring transparency and accountability in village-level governance has become a significant challenge, particularly in the management of public funds and resolution of local issues. Traditional systems often lack real-time visibility, structured communication, and reliable tracking mechanisms, leading to inefficiencies, delays, and reduced trust between citizens and authorities.

This project presents *Voice of Village: Fund Transparency & Issue Reporting System*, a web-based platform designed to enhance transparency, citizen participation, and administrative efficiency at the village level. The system enables villagers to report issues such as infrastructure problems, sanitation concerns, and public service disruptions, while also allowing authorities to track, manage, and resolve these issues in a structured manner.

The platform incorporates a role-based access control mechanism, supporting multiple stakeholders including villagers, Sarpanch (village head), administrators, and government officials. Each user is provided with specific functionalities based on their role, ensuring secure and organized interaction within the system. The issue reporting module allows users to submit complaints with descriptions and optional images, track their status, and participate through comments and feedback.

In addition to issue management, the system is designed to support transparency in fund utilization by providing clear visibility into how public funds are allocated and used for village development activities. This helps in reducing corruption, improving accountability, and building trust among stakeholders.

The backend of the system is developed using Node.js and Express.js, with MySQL as the database, while the frontend is built using React.js to provide a responsive and user-friendly interface. Overall, the system offers a scalable and efficient solution to bridge the gap between citizens and authorities, promoting transparent governance, faster issue resolution, and active community participation in village development.

LIST OF FIGURES AND SCREENSHOTS

S.no	Fig No	Title	Pg No
1.	4.1.1	System Architecture	32
2.	4.2.1	ER Diagram	33
3.	4.3.1	Use case Diagram	35
4.	4.3.2	Class Diagram	36
5.	4.3.3	Sequence Diagram	37
6.	4.3.4	Activity Diagram	38
7.	4.4.1	User Interface Design	39
8.	7.1.1	Villager	66
9.	7.1.2	villager profile	66
10.	7.1.3.1	PDS System	67
11.	7.1.3.2	PDS System	67
12.	7.1.4.1	water request	68
13.	7.1.4.2	water request	68
14.	7.1.5.1	Electricity request	69
15.	7.1.5.2	Electricity request	69
16.	7.1.6.1	House request	70
17.	7.1.6.2	House Request	70
18.	7.1.6.3	House request	70
19.	7.1.7	Issues Reporting	71
20.	7.1.8	Donations page	71
21.	7.1.9	Reports	71
22.	7.1.8.2	Reports	72

LIST OF TABLES

S.no	Table No	Title	Pg No
1.	2.4	Comparative Study	11
2.	6.5.1	Test Cases and Results	51
3.	6.6.1	Bug Report and Tracking	53

LIST OF ABBRIVATIONS

PDS – Public Distribution System

NGO – Non-Governmental Organization

CSR – Corporate Social Responsibility

SDG – Sustainable Development Goals

GP – Gram Panchayat

FG – Fund Governance

IR – Issue Reporting

VMF – Village Management Fund

TABLE OF CONTENTS

S.No	Content	Page No.
1.	Title Page	i
2.	Certificate	ii
3.	Declaration	iii
4.	Acknowledgement	iv
5.	Abstract	v
6.	List of figures	vi
7.	List of Tables	vii
8.	List of abbreviations	viii
9.	Table of Contents	ix

CHAPTER 1: INTRODUCTION

1.1	Overview of the Project	1
1.2	Problem Statement	2
1.3	Objectives of the Project	3
1.4	Scope of the Project	4
1.5	Methodology / SDLC Model Adopted	5
1.6	Organization of the Report	6

CHAPTER 2: LITERATURE SURVEY

2.1	Review of Existing System	8
2.2	Limitations of Existing Approaches	9
2.3	Need for the Proposed System	10
2.4	Comparative Study	11
2.5	Summary	13

CHAPTER 3: SYSTEM ANALYSIS

3.1 Feasibility Study 14
• Technical Feasibility
• Economic Feasibility
• Operational Feasibility
• Time & Cost Estimation

3.2 Software Requirements Specification (SRS) 16
3.3 Functional and Non-Functional Requirements 18

CHAPTER 4: SYSTEM DESIGN

4.1 System Architecture 21
4.2 Database Design 22
4.3 UML Diagrams 24
4.4 User Interface Design 28
4.5 Design Standards Followed 28
4.6 Safety & Risk Mitigation Measures 30

CHAPTER 5: IMPLEMENTATION

5.1 Technology Stack 31
5.2 Module-wise Implementation 32
5.3 Code Integration Strategy 33
5.4 Sample Code Snippets 34
5.5 Coding Standards Followed 46

CHAPTER 6: TESTING

6.1 Testing Strategy 48
6.2 Unit Testing 48
6.3 Integration Testing 49
6.4 System Testing 50
6.5 Test Cases and Results 51
6.6 Bug Reporting and Tracking 53
6.7 Quality Assurance Standards 54

CHAPTER 7: RESULTS AND DISCUSSION

7.1 Output Screenshots 55

7.2 Results Interpretation 61

7.3 Performance Evaluation 62

7.4 Comparative Results 62

CHAPTER 8: CONCLUSION AND FUTURE SCOPE

8.1 Summary of Work Done 64

8.2 Limitations 65

8.3 Challenges Faced 65

8.4 Future Enhancements 67

CHAPTER 9: REFERENCES

- Technical Publications 68
- Websites and forums details 68
- Any other sources 68

CHAPTER 10: APPENDICES

A. SDLC Forms (SRS, Feasibility, Test Reports, etc.) 69

B. Gantt Chart / Project Timeline 70

C. Ethical Considerations & Consent 70

D. Plagiarism Report (Turnitin /URKUND / DrillBit Certificate) 71

E. Source Code Repository / Deployment Links (eg. GITHUB / Drive links) 72

F. Journal / Conference paper published on project 72

G. Soft Copy / Drive Link for all project related files (All review PPTs , documentation soft copy, Video demonstration of project etc..) 72

1. Introduction

1.1 Overview of the Project

In the current era of digital transformation, there is an increasing need to improve transparency, accountability, and efficiency in local governance systems, especially at the village level. Traditional methods of managing public funds and addressing citizen issues are often manual, fragmented, and lack proper tracking mechanisms. This results in delays, miscommunication, and limited visibility for citizens regarding how decisions are made and how resources are utilized.

The *Voice of Village: Fund Transparency & Issue Reporting System* is designed to address these challenges by providing a centralized, web-based platform that connects villagers with local authorities. The system enables citizens to actively participate in governance by reporting issues related to infrastructure, sanitation, water supply, electricity, and other public services. It also ensures that these issues are properly tracked, managed, and resolved in a transparent and organized manner.

The platform follows a structured approach where each reported issue is recorded with relevant details such as title, description, category, and optional image evidence. Authorities such as the Sarpanch, administrators, and government officials can review these issues, update their status, and provide necessary actions. The inclusion of a commenting feature allows continuous interaction between stakeholders, improving communication and clarity.

A key aspect of the system is its focus on transparency in fund utilization. It aims to provide visibility into how village-level funds are allocated and used for development activities. By integrating both issue reporting and fund transparency into a single platform, the system promotes accountability and reduces the chances of corruption or misuse of resources.

The application supports multiple user roles with role-based access control, ensuring that each stakeholder interacts with the system according to their responsibilities. The backend of the system is developed using Node.js and Express.js, with MySQL as the database for efficient data management, while the frontend is built using React.js to provide a responsive and user-friendly experience. Another important feature of the Voice of Village system is the use of interactive dashboards and data visualization tools, which provide clear insights into fund allocation, expenditure, and issue resolution status. These visual representations help villagers easily understand complex financial data without requiring technical knowledge. This feature not only enhances transparency but also supports better decision-making for both citizens and authorities, ultimately leading to more efficient and responsive governance.

Overall, the Voice of Village system serves as a digital bridge between citizens and authorities, enabling efficient issue resolution, transparent fund management, and active community participation in village development.

1.2 Problem Statement

In recent years, the need for transparency and accountability in village-level governance has become increasingly important. However, most existing systems for managing public issues and tracking fund utilization are either manual or poorly digitized, leading to inefficiencies and lack of trust among citizens. Villagers often face difficulties in reporting local problems such as damaged roads, water shortages, sanitation issues, and electricity failures, as there is no structured platform to register and monitor these complaints.

One of the major problems is the absence of a centralized system that allows citizens to communicate directly with local authorities. Complaints are typically conveyed through informal channels, which can result in delays, mismanagement, or even complete neglect of issues. Additionally, there is no proper mechanism to track the status of reported problems, leaving citizens unaware of whether their concerns are being addressed.

Another significant challenge is the lack of transparency in the utilization of village development funds. Citizens have limited or no access to information regarding how funds are allocated, spent, or managed by local authorities. This lack of visibility can lead to misuse of resources, corruption, and reduced public trust in governance systems.

Furthermore, there is minimal interaction and collaboration between different stakeholders such as villagers, Sarpanch, administrators, and government officials. The absence of a unified platform results in fragmented communication, poor coordination, and inefficient decision-making.

Existing approaches also fail to provide real-time updates, structured data management, and user-friendly interfaces for both citizens and authorities. As a result, the overall governance process remains slow, non-transparent, and less effective.

Therefore, there is a need for a comprehensive and integrated system that enables efficient issue reporting, real-time tracking, transparent fund management, and seamless communication among all stakeholders. Such a system should empower citizens, improve administrative efficiency, and promote accountability in village-level governance.

1.3 Objectives of the Project

The main objective of the *Voice of Village: Fund Transparency & Issue Reporting System* is to develop a centralized and user-friendly platform that enhances transparency, accountability, and efficiency in village-level governance. The system aims to empower citizens by providing them with the tools to actively participate in reporting issues and monitoring public fund utilization.

The specific objectives of the project are as follows:

- To design and develop a web-based system that enables villagers to report local issues such as infrastructure damage, sanitation problems, water supply issues, and other public concerns in a structured manner
- To provide a mechanism for tracking the status of reported issues, allowing users to monitor progress from submission to resolution
- To implement a role-based access control system that supports different stakeholders, including villagers, Sarpanch, administrators, and government officials
- To facilitate effective communication between citizens and authorities through features such as comments and updates on reported issues
- To ensure transparency in village fund utilization by providing clear and accessible information about fund allocation and usage
- To reduce delays, miscommunication, and manual intervention by digitizing the process of issue reporting and management
- To maintain a centralized database for storing and managing all issue-related and fund-related data efficiently
- To build a secure system with authentication mechanisms to protect user data and ensure authorized access
- To create a scalable and modular platform that can be extended with additional features such as analytics, notifications, and mobile integration in the future
- To promote accountability and trust between citizens and local authorities by providing a transparent and interactive governance platform
- Overall, the project aims to bridge the gap between villagers and governing bodies by leveraging technology to create a more transparent, responsive, and participatory governance system.

1.4 Scope of the Project

The *Voice of Village: Fund Transparency & Issue Reporting System* is designed to provide a comprehensive digital solution for improving transparency, communication, and efficiency in village-level governance. The scope of the project includes the development of a web-based platform that enables multiple stakeholders to interact within a unified system.

The system covers the complete lifecycle of issue reporting, starting from submission by villagers to final resolution by authorities. Users can report issues with relevant details such as title, description, category, and optional image evidence. These issues are then managed by authorized personnel such as the Sarpanch, administrators, and government officials, who can review, update status, and take necessary actions. The system also allows users to track the progress of their complaints and engage through comments, ensuring continuous communication.

In addition to issue management, the project includes features for enhancing transparency in fund utilization. The platform is designed to provide visibility into how village development funds are allocated and used, enabling citizens to stay informed and promoting accountability among authorities.

The system supports multiple user roles, including villagers, Sarpanch, administrators, and government officials. Each role is assigned specific permissions through a role-based access control mechanism, ensuring secure and structured interaction with the system. The platform also incorporates authentication and authorization features to maintain data security and integrity.

The project includes the development of a backend system using Node.js and Express.js to handle business logic and API communication, along with a MySQL database for efficient data storage and management. The frontend is developed using React.js to provide a responsive and user-friendly interface.

However, the scope of the project is limited to a controlled environment and does not include real-time integration with external government databases, advanced analytics systems, or mobile applications. Features such as real-time notifications, geolocation-based issue tracking, and large-scale deployment are considered future enhancements.

Overall, the project demonstrates a scalable and modular approach to improving governance at the village level by integrating issue reporting and fund transparency into a single platform.

1.5 Methodology / SDLC Model Adopted

The development of the *Voice of Village: Fund Transparency & Issue Reporting System* follows the Agile Software Development Methodology. This approach was chosen due to its flexibility, iterative nature, and ability to accommodate changes during the development process. Since the project involves multiple modules such as authentication, issue reporting, fund transparency, and role-based access, Agile enables gradual development and continuous improvement of each component.

Agile methodology focuses on developing the system in small, manageable iterations called sprints. Each sprint involves planning, development, testing, and review of specific features. This allows the team to identify issues early, incorporate feedback, and enhance the system incrementally. It is especially suitable for this project as requirements can evolve based on user needs and testing outcomes.

The key aspects of the adopted methodology are as follows:

- **Iterative Development:** The system is developed in multiple phases, where core modules such as user authentication, issue reporting, and database integration are implemented step by step and refined over time
- **Continuous Feedback:** Regular feedback is incorporated during development to improve functionality, usability, and performance of the system
- **Flexibility:** The methodology allows easy modification and addition of features such as fund tracking, notifications, and enhanced reporting mechanisms
- **Parallel Development:** Frontend and backend components are developed simultaneously, ensuring efficient use of time and faster integration
- **Regular Testing:** Testing is conducted at each stage to identify bugs, ensure correctness, and maintain system stability

The Agile workflow followed in this project includes the following stages:

1. Requirement gathering and analysis
2. Sprint planning and task allocation
3. Module-wise development (authentication, issue reporting, fund tracking)
4. Testing and debugging of individual modules
5. Integration of frontend and backend components
6. Review and feedback incorporation
7. Deployment and iterative improvements

This methodology ensured that the system was developed in a structured yet adaptable manner, resulting in a reliable, user-friendly, and scalable application that meets the project objectives.

1.6 Organization of the Report

This report is organized into multiple chapters, each focusing on a specific aspect of the *Voice of Village: Fund Transparency & Issue Reporting System*, providing a clear and systematic understanding of the project from concept to implementation.

Chapter 1: Introduction: This chapter presents an overview of the project, including the problem statement, objectives, scope, and the methodology adopted for development.

Chapter 2: Literature Survey: This chapter reviews existing systems related to issue reporting and governance transparency. It discusses their limitations, the need for the proposed system, and provides a comparative analysis.

Chapter 3: System Analysis: This chapter focuses on the feasibility study, including technical, economic, and operational feasibility. It also includes the Software Requirements Specification (SRS) and identifies functional and non-functional requirements of the system.

Chapter 4: System Design: This chapter describes the overall system design, including system architecture, database design, UML diagrams, user interface design, and design standards followed.

Chapter 5: Implementation: This chapter explains the technologies used and provides details about module-wise implementation, code integration strategy, and coding standards followed during development.

Chapter 6: Testing: This chapter discusses the testing strategies adopted, including unit testing, integration testing, system testing, and quality assurance practices. It also includes test cases and bug tracking details.

Chapter 7: Results and Discussion: This chapter presents the output of the system with screenshots, analysis of results, and evaluation of system performance.

Chapter 8: Conclusion and Future Scope: This chapter summarizes the work done, highlights the limitations and challenges faced, and suggests possible future enhancements.

Chapter 9: References: This chapter lists all the sources referred to during the project, including technical papers, websites, and other materials.

Chapter 10: Appendices: This chapter includes supporting documents such as SRS forms, test reports, project timeline, ethical considerations, plagiarism report, source code links, and other relevant materials.

This structured organization ensures that the report is comprehensive, logically arranged, and easy to understand for readers and evaluators.

2. Literature Survey

2.1 Review of Existing System

Existing systems in the domain of governance and public issue management primarily focus on either complaint registration or administrative record-keeping, but rarely provide a complete and integrated solution. Traditional approaches in villages largely rely on manual methods such as written applications, in-person reporting to local authorities, or informal communication channels. These methods are time-consuming, lack proper documentation, and often result in delays or loss of information.

In recent years, some digital platforms and e-governance systems have been introduced to improve communication between citizens and authorities. These systems allow users to submit complaints online and enable officials to manage and track them. Such platforms help in organizing data, improving response time, and maintaining records of reported issues. However, their implementation at the village level is still limited, and many of these systems are designed for urban or large-scale administrative use.

Issue tracking systems used in organizations also provide structured workflows for reporting, assigning, and resolving problems. These systems include features such as status updates, comments, and notifications, which enhance coordination among users. While they are efficient in controlled environments, they are not specifically designed to address the unique challenges of rural governance, such as accessibility, simplicity, and multi-role participation.

Additionally, some transparency initiatives focus on publishing government data related to budgets and expenditures. These systems aim to improve accountability by making financial information available to the public. However, they often lack user-friendly interfaces and do not provide real-time or detailed insights into how funds are utilized at the village level.

Overall, existing systems offer partial solutions by addressing specific aspects such as issue reporting or data transparency. However, they operate independently and lack integration, making it difficult to achieve complete transparency, efficient issue resolution, and effective communication among all stakeholders in a village ecosystem.

2.2 Limitations of Existing Approaches

Despite the availability of various systems for issue reporting and governance management, several limitations still exist in current approaches, especially at the village level.

One of the primary limitations is the lack of integration. Most existing systems focus on a single aspect, such as complaint registration or data publication, without providing a unified platform that combines issue tracking, communication, and fund transparency. This results in fragmented data and inefficient coordination among stakeholders.

Another major drawback is the dependence on manual or semi-digital processes. In many villages, issues are still reported through written applications or verbal communication, which are not properly recorded or tracked. This leads to delays, misplacement of complaints, and lack of accountability in resolving issues.

Existing digital systems often lack transparency and verification mechanisms. Information provided by authorities is not always accessible or easy to understand for citizens, and there is limited visibility into how decisions are made or how funds are utilized. This creates a gap between citizens and governing bodies, reducing trust.

Additionally, most systems do not provide real-time tracking of issues. Once a complaint is submitted, citizens are often unaware of its progress or status. The absence of status updates and feedback mechanisms limits user engagement and satisfaction.

Another limitation is the lack of role-based interaction. Many systems do not clearly define responsibilities for different stakeholders such as villagers, Sarpanch, administrators, and government officials. This results in confusion, overlapping tasks, and inefficient handling of issues.

Furthermore, user accessibility and interface design are often neglected. Existing platforms may be complex, not mobile-friendly, or not tailored for users with limited technical knowledge, making them difficult to use in rural environments.

Finally, transparency in fund utilization is either limited or completely absent in most systems. Citizens do not have access to clear and detailed information about how public funds are allocated and spent, which increases the risk of misuse and corruption.

These limitations highlight the need for a more comprehensive, user-friendly, and integrated system that can effectively address issue reporting, tracking, communication, and fund transparency in a single platform.

2.3 Need for the Proposed System

The limitations of existing systems highlight the urgent need for a comprehensive and integrated solution that can improve transparency, accountability, and efficiency in village-level governance. A system is required that not only allows citizens to report issues but also ensures proper tracking, communication, and resolution in a structured and reliable manner.

There is a strong need for a centralized platform where all stakeholders—villagers, Sarpanch, administrators, and government officials—can interact within a single system. Such a platform should eliminate fragmented communication and provide a unified view of all reported issues and administrative actions.

Another important requirement is the ability to track issues in real time. Citizens should be able to monitor the status of their complaints from submission to resolution. This increases transparency, keeps users informed, and encourages authorities to respond more efficiently.

The system must also incorporate a clear and structured mechanism for communication. Features such as comments, updates, and feedback are essential to ensure proper interaction between citizens and authorities. This helps in clarifying issues, providing updates, and improving overall coordination.

A key need is to enhance transparency in fund utilization. Citizens should have access to information regarding how public funds are allocated and used for development activities. This reduces the chances of corruption and builds trust in the governance system.

Additionally, there is a requirement for role-based access control to ensure that each stakeholder has clearly defined responsibilities and permissions. This helps in maintaining order, improving accountability, and preventing unauthorized actions.

The system should also be user-friendly and accessible, especially for users with limited technical knowledge. A simple and intuitive interface is necessary to encourage adoption and ensure effective usage in rural environments.

Therefore, the proposed *Voice of Village: Fund Transparency & Issue Reporting System* is needed to address these challenges by providing an integrated platform that combines issue reporting, real-time tracking, transparent fund management, and efficient communication, ultimately improving governance and citizen participation at the village level.

2.4 Comparative Study

A comparative analysis between existing systems and the proposed Voice of Village: Fund Transparency & Issue Reporting System highlights the improvements and advantages offered by the proposed approach.

Feature	**Existing Systems**	**EcoTrace System**
Issue Reporting	Available but often manual or unstructured	Structured digital issue reporting with detailed inputs
Issue Tracking	Limited or no tracking	Real-time status tracking (pending, in progress, resolved)
Data Integration	Fragmented across different systems	Centralized platform with unified data

Communication	Minimal or informal communication	Comment-based interaction between users and authorities
Transparency	Limited visibility of processes	High transparency in issue handling and updates
Fund Utilization	Rarely included or not accessible	Clear visibility of fund allocation and usage
User Roles	Not clearly defined	Role-based access (villager, Sarpanch, admin, government)
Accessibility	Complex or not user-friendly	Simple, intuitive, and user-friendly interface
Accountability	Low due to lack of tracking	Improved accountability through tracking and records

Table 2.4 Comparative Study

2.5 Summary

This chapter presented a detailed study of existing systems related to issue reporting, governance management, and transparency mechanisms. It highlighted how traditional and current digital approaches provide partial solutions, focusing either on complaint registration, administrative processes, or data publication, but failing to deliver a complete and integrated system.

The limitations identified include lack of data integration, dependence on manual processes, absence of real-time tracking, poor communication mechanisms, limited user accessibility, and lack of transparency in fund utilization. These issues reduce efficiency, accountability, and trust between citizens and authorities.

Based on these observations, the need for a unified and user-friendly system was established. The proposed *Voice of Village: Fund Transparency & Issue Reporting System* addresses these gaps by integrating issue reporting, tracking, communication, and fund transparency into a single platform. It provides role-based access, real-time updates, and improved interaction among stakeholders.

Overall, this chapter justifies the importance of the proposed system and sets the foundation for the subsequent chapters, which focus on system analysis, design, implementation, and evaluation.

3. System Analysis

3.1 Feasibility Study

The feasibility study evaluates the practicality and viability of developing and implementing the *Voice of Village: Fund Transparency & Issue Reporting System*. It examines whether the proposed system is technically possible, economically viable, operationally effective, and achievable within the given time and cost constraints.

Technical Feasibility

The proposed system is technically feasible as it is built using widely used and well-supported technologies. The backend is developed using Node.js and Express.js, which provide efficient handling of server-side operations and API development. The frontend is implemented using React.js, enabling a responsive and interactive user interface.

The system uses MySQL as the database for structured data storage and management. Authentication and authorization are implemented using JSON Web Tokens (JWT), ensuring secure access to the system. Additional features such as image uploads, issue tracking, and role-based access control are implemented using standard libraries and frameworks.

All required tools and technologies are readily available, open-source, and compatible with standard development environments. The system does not require specialized hardware and can run on general-purpose computers, making it technically practical to develop and deploy.

Economic Feasibility

The project is economically feasible as it primarily uses open-source technologies, which significantly reduce development and operational costs. Tools such as Node.js, React.js, and MySQL are freely available and widely supported.

The system does not require expensive infrastructure or proprietary software. It can be deployed on low-cost cloud platforms or local servers, minimizing hosting expenses.

Maintenance costs are also manageable due to the modular design and use of standard technologies.

Overall, the project can be developed within a reasonable budget, making it cost-effective for implementation at the village level.

Operational Feasibility

The system is operationally feasible as it is designed to be simple, user-friendly, and accessible to all stakeholders, including villagers with limited technical knowledge. The interface is intuitive, allowing users to easily report issues, track progress, and interact with authorities.

Role-based access ensures that each user—villager, Sarpanch, administrator, and government official—can perform only the actions relevant to their responsibilities. This improves system efficiency and reduces confusion.

The system enhances communication, reduces manual effort, and improves transparency, making it practical and beneficial for real-world usage.

Time & Cost Estimation

The development of the system is achievable within the given academic timeline by following an Agile methodology. The project is divided into modules such as authentication, issue reporting, database integration, and frontend development, which are implemented in phases.

Since the project uses open-source tools, the overall cost is minimal and mainly involves development time and basic infrastructure requirements. Proper planning and modular development ensure timely completion of the project.

Overall, the feasibility study confirms that the proposed system is practical, cost-effective, and suitable for implementation, with the potential to significantly improve transparency and efficiency in village governance.

3.2 Software Requirements Specification (SRS)

The Software Requirements Specification (SRS) defines the overall requirements of the Voice of Village: Fund Transparency & Issue Reporting System. It describes the system's functionality, user interactions, and constraints, serving as a foundation for system design and implementation.

1. System Overview

The system is a web-based application designed to improve transparency and communication in village governance. It enables users to report issues, track their progress, and view fund utilization details. The platform supports multiple user roles, including villagers, Sarpanch, administrators, and government officials, each with specific functionalities.

2. User Classes and Characteristics

- **Villager/Citizen:** Can register, log in, report issues, view issue status, and add comments. Basic technical knowledge is sufficient.
- **Sarpanch (Village Head)**: Responsible for reviewing reported issues, updating their status, and managing local governance activities.
- **Administrator:** Manages users, monitors system activity, and ensures proper functioning of the platform.
- **Government Official:** Oversees issue resolution, monitors progress, and ensures accountability in governance processes.

3. Functional Requirements

- User registration and login with secure authentication
- Role-based access control for different users
- Issue reporting with details such as title, description, category, and optional image
- Viewing and tracking the status of reported issues
- Updating issue status by authorized users (pending, in progress, resolved)
- Adding and viewing comments on issues for communication

- Centralized storage and retrieval of issue data
- Fund transparency module to display allocation and usage details
- Secure API communication between frontend and backend

4. Non-Functional Requirements

- **Performance:** The system should respond quickly to user requests and handle multiple users efficiently
- **Usability:** The interface should be simple, intuitive, and easy to use for all types of users
- **Security:** Data should be protected using authentication (JWT) and proper access control mechanisms
- **Scalability:** The system should support future enhancements and increased user load
- **Reliability:** The system should function consistently with minimal errors or downtime
- **Maintainability:** The code should be modular and easy to update or extend

5. System Constraints

- The system is dependent on internet connectivity for operation
- It is designed for a controlled environment and does not integrate with external government databases
- Advanced features such as real-time notifications and mobile applications are not included in the current scope

6. Assumptions and Dependencies

- Users have access to a device with internet connectivity
- Basic knowledge of using web applications is assumed
- The system relies on standard technologies such as Node.js, React.js, and MySQL

Overall, the SRS provides a clear understanding of system requirements and serves as a blueprint for designing and developing the application effectively.

3.3 Functional and Non-Functional Requirements

This section describes the detailed functional and non-functional requirements of the Voice of Village: Fund Transparency & Issue Reporting System. These requirements define what the system should do and how it should perform.

Functional Requirements

Functional requirements specify the core functionalities that the system must provide.

- **User Registration and Login:**

  The system should allow users to register and login securely using email and pass credentials**.**

- **Role-Based Access Control:**

  The system should support multiple roles such as villager, Sarpanch, administrator, and government official, with specific permissions for each role.

- **Issue Reporting:**

  Users should be able to submit issues with details such as title, description, category, and optional image upload.

- **Issue Tracking:**

  The system should allow users to view and track the status of their reported issues (e.g., pending, in progress, resolved).

- **Issue Management:**

  Authorized users should be able to update the status of issues and manage their resolution process.

- **Comment System**:

Users should be able to add comments to issues to facilitate communication and provide updates.

- **Data Management**:

  The system should store and retrieve all data related to users, issues, and comments in a structured database.

- **Fund Transparency (Conceptual Module):**

  The system should provide information about fund allocation and usage to improve transparency.

- **API Integration**:
  The frontend should communicate with the backend through RESTful APIs.

Non-Functional Requirements

Non-functional requirements define the quality attributes and constraints of the system.

- **Performance:**
  The system should provide fast response times and handle multiple user requests efficiently.

- **Usability:**
  The interface should be simple, intuitive, and easy to navigate, even for users with minimal technical knowledge.

- **Security:**
  The system should ensure secure authentication using JWT and protect sensitive user data from unauthorized access.

- **Scalability:s**
  The system should be designed to handle future growth in users and additional features.

- **Reliability:**
  The system should operate consistently without frequent failures or errors.

- **Maintainability:**

The codebase should be modular and well-structured to allow easy updates and maintenance.

- **Compatibility:**
  The system should work across different devices and modern web browsers.

Overall, these requirements ensure that the system is not only functionally complete but also efficient, secure, and user-friendly, meeting the needs of both citizens and authorities.

4. System Design

4.1 System Architecture

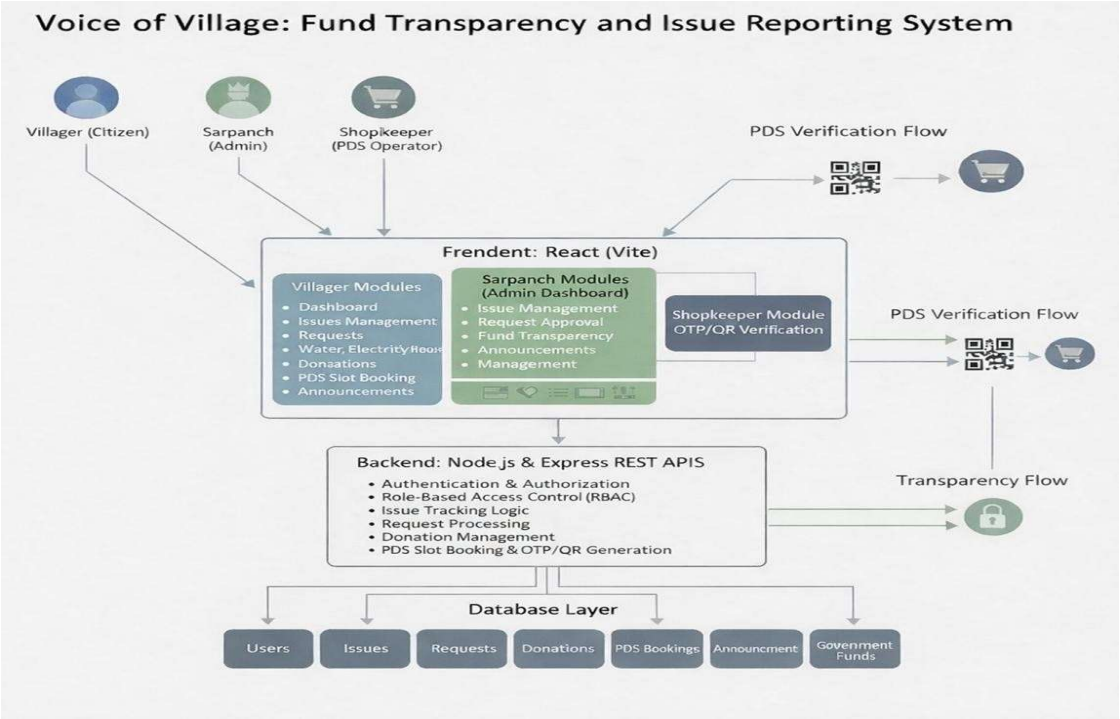

Fig 4.1.1 System Architecture

The *Voice of Village: Fund Transparency & Issue Reporting System* follows a three-tier architecture consisting of the frontend, backend, and database. This structure ensures proper separation of concerns and efficient system operation.

The frontend is developed using React.js and provides a user-friendly interface for interaction. The backend is built using Node.js and Express.js, which handle business logic, API requests, and authentication. The database uses MySQL to store and manage all system data such as users, issues, and comments.

The system follows a client-server model where the frontend communicates with the backend through APIs, and the backend interacts with the database to process and retrieve data. This architecture ensures scalability, maintainability, and smooth communication between system components.

4.2 Database Design (ER Diagram) / Data Flow Diagrams / Workflow

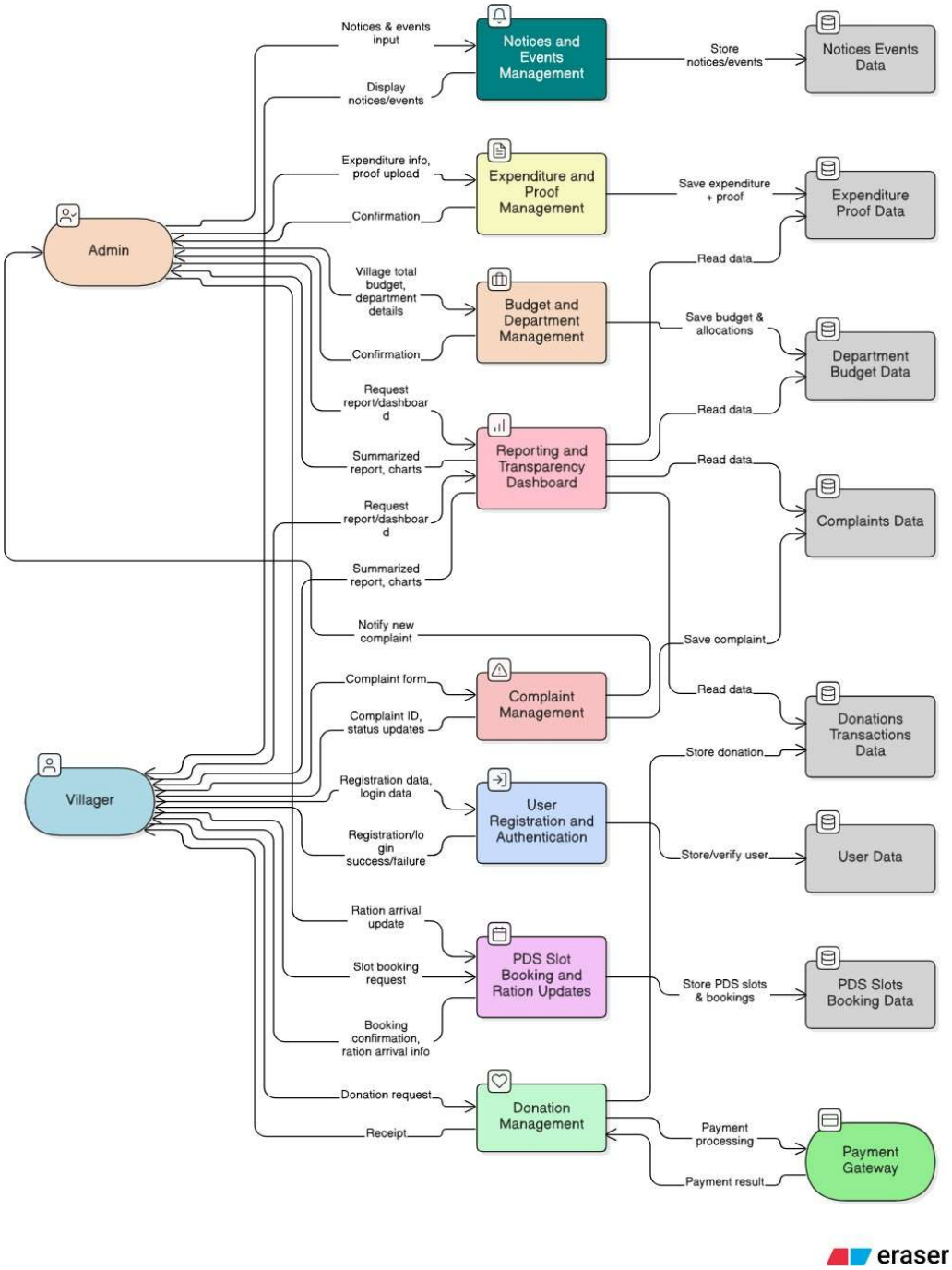

Fig 4.2.1 ER Diagram

The database design of the *Voice of Village: Fund Transparency & Issue Reporting System* is based on a relational model using MySQL. It is structured to efficiently store and manage data related to users, issues, and comments while maintaining data integrity and consistency.

The system consists of three main tables: **Users**, **Issues**, and **Issue Comments**.

The **Users** table stores user-related information such as name, email, password, role, and account creation details. It supports multiple roles including villager, Sarpanch, administrator, and government official.

The **Issues** table contains details of all reported issues, including title, description, category, status, image reference, and the user who submitted the issue. Each issue is linked to a user through a foreign key relationship.

The **Issue Comments** table stores comments made on issues by different users. It includes fields such as comment text, associated issue ID, user ID, and timestamp. This enables communication and discussion on each reported issue.

Relationships are established between these tables using primary and foreign keys to ensure referential integrity. The design avoids redundancy and supports efficient data retrieval and updates.

Overall, the database is designed to be simple, structured, and scalable, allowing easy management of data and supporting future enhancements.

4.3 UML Diagrams (Use Case, Class, Sequence, Activity)

4.3.1 Use Case Diagram

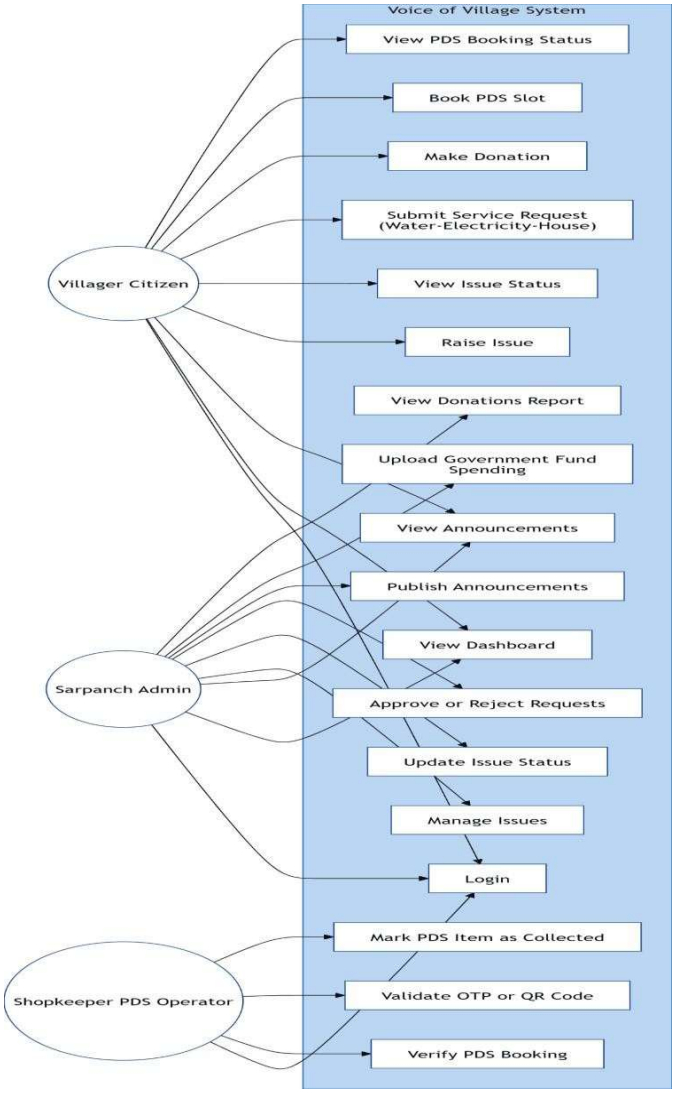

Fig 4.3.1 Use Case Diagram

The Use Case Diagram shows how different users interact with the system. The main actors are Villager, Sarpanch, Administrator, and Government Official.

Villagers can register, log in, report issues, view status, and add comments. Authorities like Sarpanch and officials can manage and update issues. Administrators oversee the system. It provides a simple view of system functionalities and user interactions.

4.3.2 Class Diagram

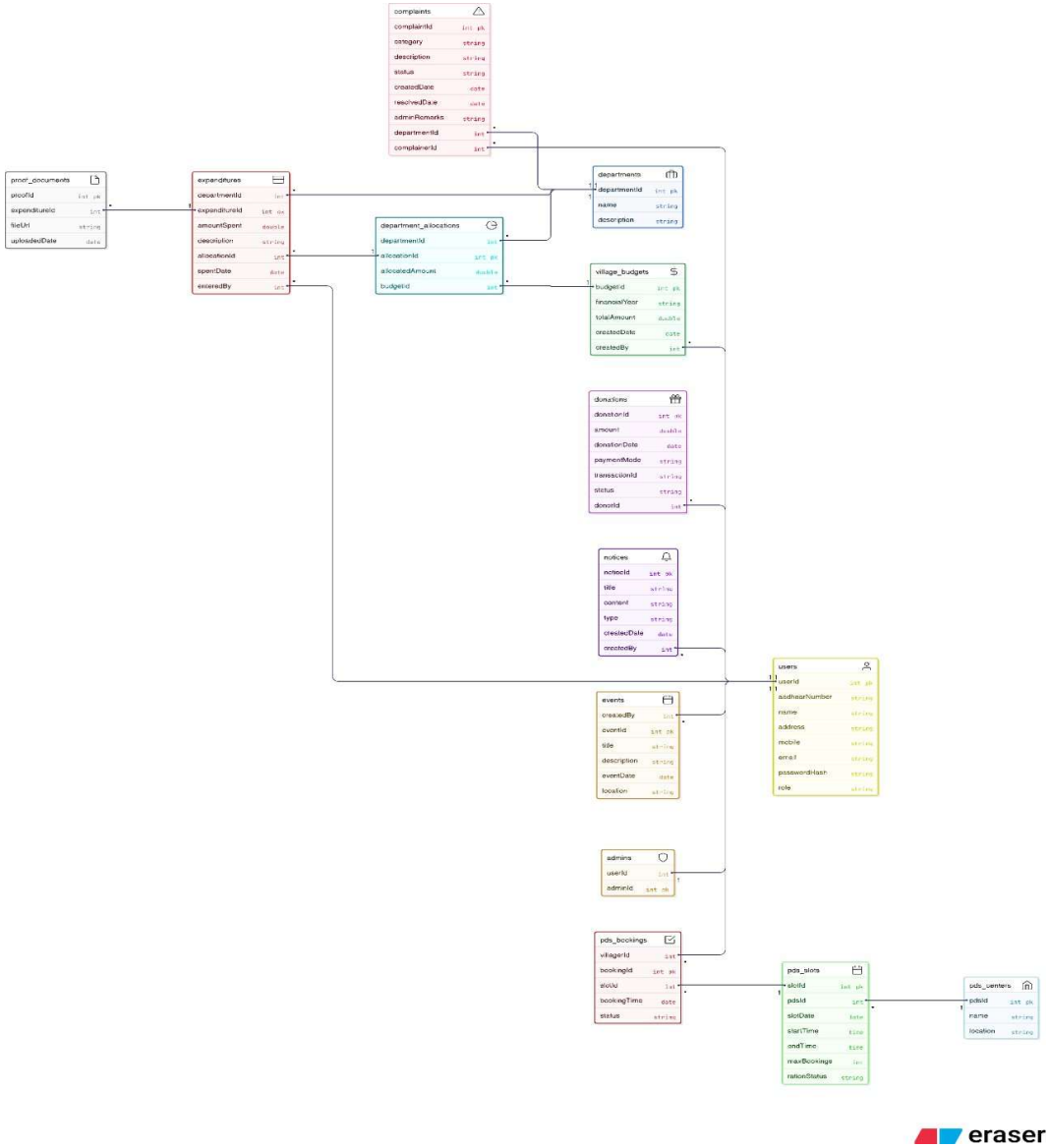

Fig 4.3.2 Class Diagram

The Class Diagram represents the structure of the system by showing different classes and their relationships. The main classes include User, Issue, and Comment. The User

class stores user details and roles. The Issue class contains information about reported issues. The Comment class manages user comments on issues. These classes are connected through relationships such as one user creating multiple issues and comments.

4.3.3 Sequence Diagram

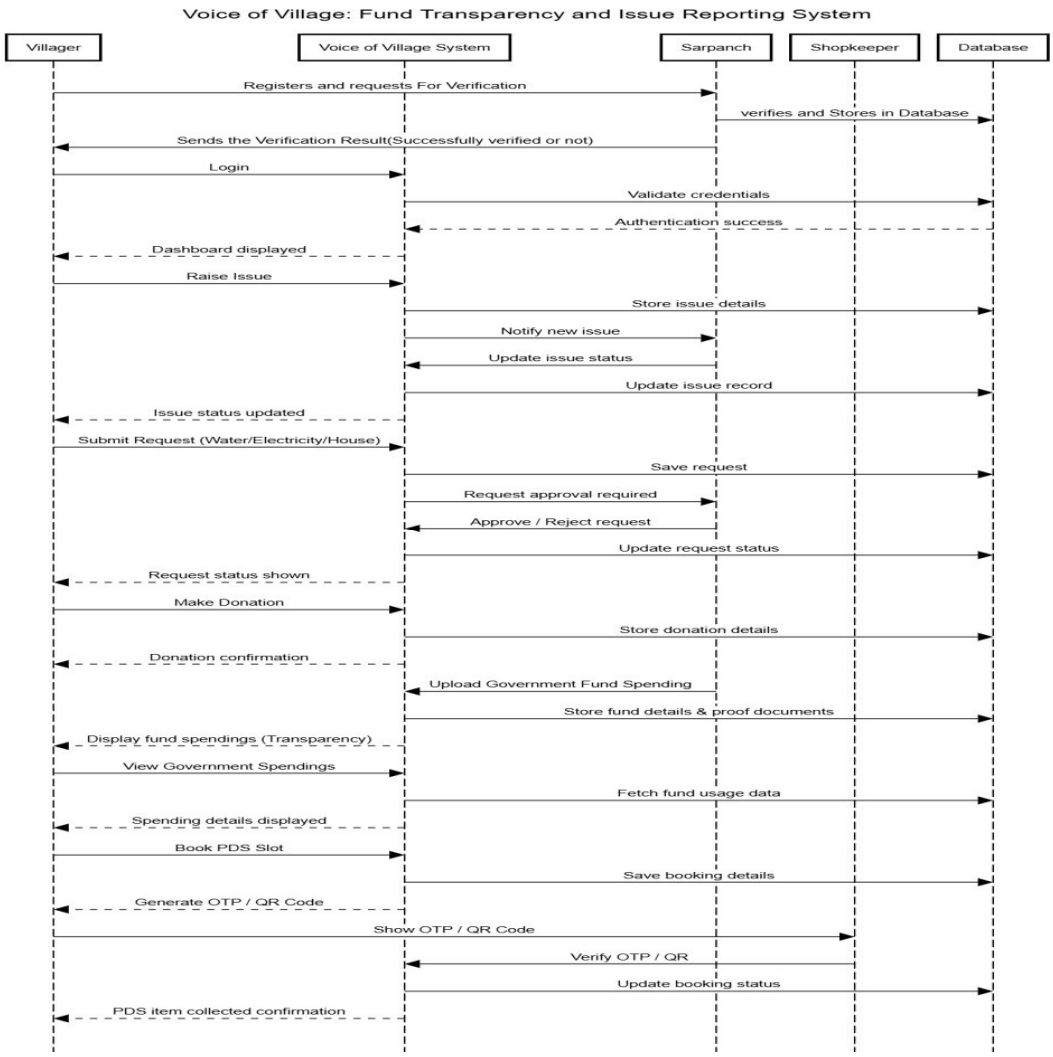

Fig 4.3.3 Sequence Diagram

The Sequence Diagram shows the flow of interactions between the user and the system over time. It illustrates how a user sends a request, the frontend forwards it to the backend, the backend processes it with the database, and then returns the response to the user. It helps in understanding the step-by-step execution of system operations.

4.3.4 Activity Diagram

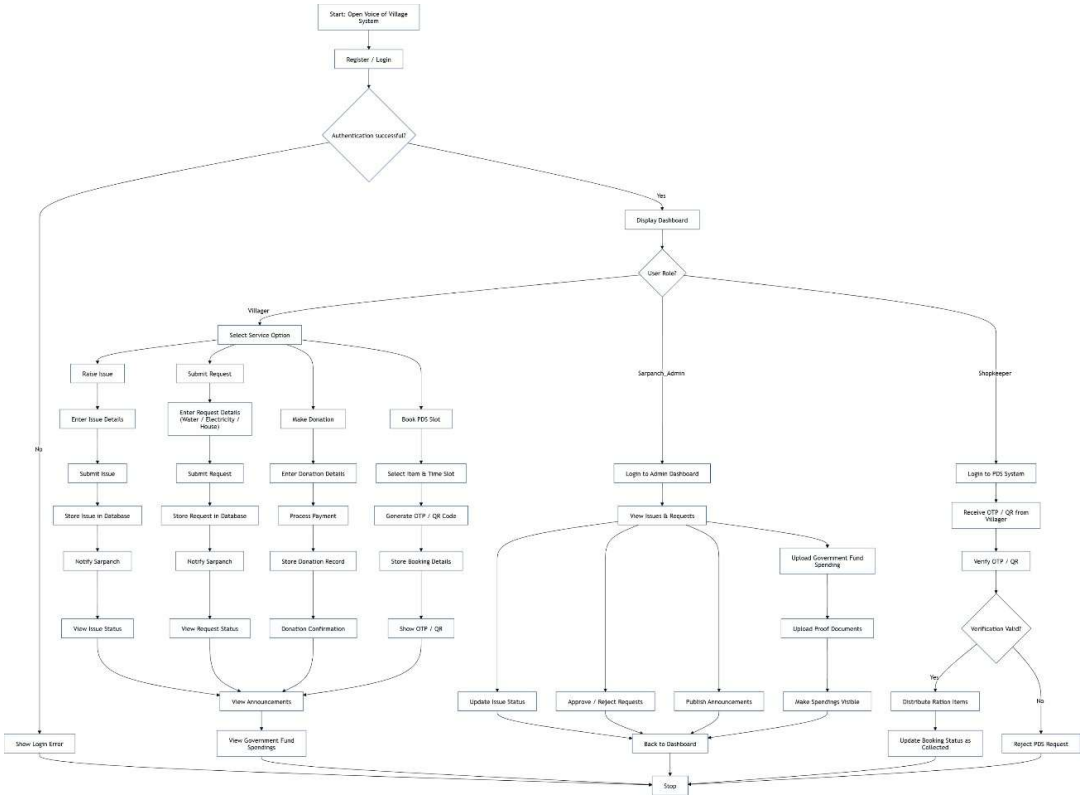

Fig 4.3.4 Activity Diagram

The Activity Diagram represents the workflow of the system. It shows the sequence of activities such as user login, issue submission, processing by authorities, and updating the issue status.

4.4 User Interface Design

Fig 4.4 User Interface Design

4.5 Design Standards Followed

The design of the *Voice of Village: Fund Transparency & Issue Reporting System* follows established software engineering standards to ensure consistency, scalability, maintainability, and ease of understanding. Adhering to these standards helps in developing a robust and well-structured system.

The system follows a **modular design approach**, where the frontend, backend, and database are separated into distinct layers. This separation of concerns improves maintainability and allows independent development and updates of each component. The backend is organized using a structured architecture with controllers, routes, middleware, and configuration files, ensuring clean code organization.

Standard **coding conventions** are followed throughout the project. Meaningful and descriptive names are used for variables, functions, and database tables, making the code more readable and understandable. Consistent indentation, formatting, and commenting practices are maintained to improve code clarity and ease of debugging.

The **user interface design** follows consistency in layout, color schemes, and navigation patterns. This ensures a smooth and intuitive user experience, especially for users with limited technical knowledge. Forms, buttons, and input fields are designed to be simple and user-friendly.

The system also follows **RESTful API design principles**, where clear and structured endpoints are defined for communication between the frontend and backend. This improves scalability and makes the system easier to integrate with other services in the future.

In terms of **database design standards**, normalization techniques are applied to reduce redundancy and maintain data integrity. Proper use of primary keys, foreign keys, and constraints ensures consistency and reliability of stored data.

Security standards are also considered, including the use of **JWT-based authentication**, password hashing, and controlled access through role-based authorization. These measures help protect sensitive data and ensure secure system operation.

Overall, following these design standards ensures that the system is well-organized, efficient, and scalable, making it easier to maintain and extend in the future.

4.6 Safety and Risk Mitigation Measures

The *Voice of Village: Fund Transparency & Issue Reporting System* incorporates several safety and risk mitigation measures to ensure secure, reliable, and efficient operation.

These measures are essential to protect user data, maintain system integrity, and prevent misuse.
One of the primary measures is **secure authentication and authorization**. The system uses JWT-based authentication to verify user identity, and role-based access control ensures that users can only perform actions permitted for their role. This prevents unauthorized access to sensitive functionalities.

To protect user data, **passwords are encrypted using hashing techniques**, ensuring that sensitive information is not stored in plain text. This enhances data security and reduces the risk of data breaches.

The system also includes **input validation and error handling mechanisms** to prevent invalid or malicious data from being processed. This helps in avoiding issues such as SQL injection and ensures the stability of the application.

Data integrity is maintained through proper database design using primary and foreign keys, ensuring that relationships between data are consistent and reliable. Regular validation checks help prevent inconsistencies.

In terms of operational safety, the system is designed to handle errors gracefully by providing appropriate error messages and preventing system crashes. Logging mechanisms can be used to track system activities and identify potential issues.
Additionally, the platform minimizes risks of misuse by maintaining transparency in actions such as issue updates and comments, which can be tracked and monitored by administrators.

Overall, these safety and risk mitigation measures ensure that the system operates securely, reliably, and efficiently, providing a trustworthy platform for users and authorities.

5. Implementation

5.1 Technology Stack

The *Voice of Village: Fund Transparency & Issue Reporting System* is developed using a simple and efficient full-stack setup.

• **Backend**

Node.js with Express.js for building APIs and handling business logic

• **Database**

MySQL for structured data storage and management

• **Frontend**

React.js for building the user interface, along with CSS for styling and React Router for navigation

• **API Communication**

Axios for connecting the frontend and backend through HTTP requests

• **Authentication**

JWT (JSON Web Tokens) for secure login and role-based access control

•**Features**

Issue reporting module for submitting and tracking complaints Comment system for communication between users and authorities Role-based system for managing different user access levels

• **Tools**

Git and GitHub for version control

Deployment

The system is deployed using cloud-based platforms to ensure accessibility and ease of use.

• **Backend Deployment**

The backend can be deployed on platforms like Render or similar services, which

support Node.js applications and provide API hosting with environment configuration.

• **Frontend Deployment**
The frontend application can be deployed on platforms like Vercel, which provide fast and efficient hosting for React applications.

• **Integration**
The frontend communicates with the backend through deployed API endpoints. Environment variables are used to manage API URLs and maintain secure communication between components.

.

5.2 Module-wise Implementation

The *Voice of Village: Fund Transparency & Issue Reporting System* is developed by dividing the application into multiple modules. Each module handles a specific functionality of the system, ensuring better organization and maintainability.

• **Authentication Module**: This module handles user registration and login. It verifies user credentials, encrypts passwords, and generates JWT tokens for secure access. It also manages role-based authentication for different users.

• **User Management Module**: This module manages user details and roles such as villager, Sarpanch, administrator, and government official. It ensures that each user has appropriate permissions within the system.

• **Issue Reporting Module**: This is the core module of the system. It allows users to submit issues with details like title, description, category, and optional images. The data is stored in the database for further processing.

• **Issue Management Module**: This module allows authorized users to view all reported issues and update their status (pending, in progress, resolved). It ensures proper tracking and resolution of issues.

• **Comment Module**: This module enables users to add comments on issues. It helps in communication between citizens and authorities and provides updates on issue progress.

• **Fund Transparency Module**: This module is designed to provide visibility into fund allocation and usage. It allows users to view how funds are utilized for development activities.

• **Database Module**: This module manages all data operations, including storing, retrieving, and updating user, issue, and comment data using MySQL.

• **API Module**: This module handles communication between frontend and backend through RESTful APIs. It ensures smooth data exchange and system functionality.

Overall, the modular approach ensures that each component of the system is well-structured, making the application easier to develop, test, and maintain.

5.3 Code Integration Strategy

The *Voice of Village: Fund Transparency & Issue Reporting System* follows a structured code integration strategy to ensure smooth interaction between different modules and components of the system. The integration process is designed to maintain consistency, reduce errors, and enable efficient development.

The system follows a **modular integration approach**, where each module such as authentication, issue reporting, and comments is developed independently and later integrated into the main application. This approach helps in isolating errors and simplifies debugging.

The **backend integration** is organized using a layered structure that includes routes, controllers, middleware, and database configuration. Routes handle incoming API requests, controllers process the business logic, middleware manages authentication and validation, and the database layer handles data operations. All these components are connected to ensure seamless execution of functionalities.

The **frontend and backend integration** is achieved through RESTful APIs. The frontend sends HTTP requests using Axios, and the backend responds with appropriate data.

Proper API endpoints are defined for each operation such as login, issue submission, fetching issues, and adding comments.

To maintain consistency, **environment variables** are used for configuration details such as database credentials and API URLs. This ensures flexibility and security during development and deployment.

Version control is managed using **Git and GitHub**, where code changes are tracked and integrated systematically. This helps in maintaining different versions of the project and enables collaborative development.

Testing is performed after integrating each module to ensure that all components work together correctly. Any issues identified during integration are resolved before moving to the next stage.

Overall, the integration strategy ensures that all parts of the system work cohesively, resulting in a stable and efficient application.

5.4 Sample Code Snippets

--------#Main Entry Point---------

```
const express = require('express');
const cors = require('cors');
require('dotenv').config();

const app = express();

app.use(cors());
app.use(express.json());

// Routes
const authRoutes = require('./routes/authRoutes');
const issueRoutes = require('./routes/issueRoutes');
```

```
const adminRoutes = require('./routes/adminRoutes');
const pdsRoutes = require('./routes/pdsRoutes');
const profileRoutes = require('./routes/profileRoutes');
const waterRoutes = require('./routes/waterRoutes');
const electricityRoutes = require('./routes/electricityRoutes');
const houseRoutes = require('./routes/houseRoutes');

app.use('/api/auth', authRoutes);
app.use('/api/issues', issueRoutes);
app.use('/api/admin', adminRoutes);
app.use('/api/pds', pdsRoutes);
app.use('/api/profile', profileRoutes);
app.use('/api/water', waterRoutes);
app.use('/api/electricity', electricityRoutes);
app.use('/api/house', houseRoutes);

const donationRoutes = require('./routes/donationRoutes');
const reportRoutes = require('./routes/reportRoutes');
const fundRoutes = require('./routes/fundRoutes');
const announcementRoutes = require('./routes/announcementRoutes');

app.use('/api/donations', donationRoutes);
app.use('/api/reports', reportRoutes);
app.use('/api/funds', fundRoutes);
app.use('/api/announcements', announcementRoutes);

// Serve static images from uploads directory
app.use('/uploads', express.static('uploads'));
```

```
app.get('/', (req, res) => {
  res.send('Voice of Village Backend is Running!');
});
------#Issue Controller--------
const PORT = process.env.PORT || 5000;
app.listen(PORT, () => {
  console.log(`Server running on port ${PORT}`);
});
const db = require('../config/db');

// Submit an issue
const submitIssue = async (req, res) => {
  const { title, description, category } = req.body;
  const submitted_by = req.user.id;
  const image_url = req.file ? req.file.filename : null;

  try {
    await db.query(
      'INSERT INTO issues (title, description, category, submitted_by, image_url)
VALUES (?, ?, ?, ?, ?)',
      [title, description, category, submitted_by, image_url]
    );
    res.status(201).json({ message: 'Issue submitted successfully' });
  } catch (err) {
    res.status(500).json({ message: 'Error submitting issue', error: err.message });
  }
};
```

```
// Get all issues
const getAllIssues = async (req, res) => {
  try {
    const [issues] = await db.query(
      'SELECT issues.*, users.name as submitted_by_name FROM issues JOIN users ON issues.submitted_by = users.id ORDER BY issues.created_at DESC'
    );
    res.status(200).json(issues);
  } catch (err) {
    res.status(500).json({ message: 'Error fetching issues', error: err.message });
  }
};

// Get single issue by ID
const getIssueById = async (req, res) => {
  const { id } = req.params;
  try {
    const [issue] = await db.query(
      'SELECT issues.*, users.name as submitted_by_name FROM issues JOIN users ON issues.submitted_by = users.id WHERE issues.id = ?',
      [id]
    );
    if (issue.length === 0) return res.status(404).json({ message: 'Issue not found' });
    res.status(200).json(issue[0]);
  } catch (err) {
    res.status(500).json({ message: 'Error fetching issue', error: err.message });
  }
};
```

```
// Update issue status (sarpanch/admin only)
const updateIssueStatus = async (req, res) => {
const { id } = req.params;
  const { status } = req.body;

  try {
    await db.query('UPDATE issues SET status = ? WHERE id = ?', [status, id]);
    res.status(200).json({ message: 'Issue status updated' });
  } catch (err) {
    res.status(500).json({ message: 'Error updating status', error: err.message });
  }
};

// Add comment to issue
const addComment = async (req, res) => {
  console.log('Body received:', req.body);
  console.log('User:', req.user);
  console.log('Issue ID:', req.params.id);

  const { comment } = req.body;
  const user_id = req.user.id;
  const issue_id = req.params.id;

  if (!comment) {
    return res.status(400).json({ message: 'Comment is required' });
  }
```

```
  try {
    await db.query(
      'INSERT INTO issue_comments (issue_id, user_id, comment) VALUES (?, ?, ?)',
      [issue_id, user_id, comment]
    );
    res.status(201).json({ message: 'Comment added successfully' });
  } catch (err) {
    console.log('DB Error:', err.message);
    res.status(500).json({ message: 'Error adding comment', error: err.message });
  }
};
// Get comments for an issue
const getComments = async (req, res) => {
  const { id } = req.params;
  try {
    const [comments] = await db.query(
      'SELECT issue_comments.*, users.name as commented_by FROM issue_comments
JOIN users ON issue_comments.user_id = users.id WHERE issue_id = ? ORDER BY
issue_comments.created_at ASC',
      [id]
    );
    res.status(200).json(comments);
  } catch (err) {
    res.status(500).json({ message: 'Error fetching comments', error: err.message });
  }
};


module.exports = { submitIssue, getAllIssues, getIssueById, updateIssueStatus,
addComment, getComments };
```

Frontend:

```
import { useState } from "react";

import { useNavigate } from "react-router-dom";

import "./ReportIssue.css";


export default function ReportIssue() {

  const [formData, setFormData] = useState({

    title: "",

    category: "",

    description: "",

    location: "",

    image: null,

  });

  const [submitting, setSubmitting] = useState(false);

  const [error, setError] = useState("");

  const [success, setSuccess] = useState("");

  const navigate = useNavigate();


  const handleChange = (e) => {

    const { name, value } = e.target;

    setFormData({ ...formData, [name]: value });
```

```
};

const handleFileChange = (e) => {
  setFormData({ ...formData, image: e.target.files[0] });
};

const handleSubmit = async (e) => {
  e.preventDefault();
  setSubmitting(true);
  setError("");
  setSuccess("");

  const data = new FormData();
  data.append("title", formData.title);
  data.append("category", formData.category);
  data.append("description", formData.description);
  data.append("location", formData.location);

  if (formData.image) {
    data.append("image", formData.image);
  }
```

```
try {
  const token = localStorage.getItem("token");
  const response = await fetch("http://localhost:5000/api/issues/submit", {
    method: "POST",
    headers: {
      "Authorization": `Bearer ${token}`
    },
    body: data,
  });

  const result = await response.json();

  if (response.ok) {
    setSuccess("Issue submitted successfully! Redirecting...");
    setTimeout(() => navigate("/issues"), 1500);
  } else {
    setError(result.message || "Failed to report issue. Please try again.");
  }
} catch (err) {
  console.error(err);
  setError("Server error. Make sure you are logged in and try again.");
} finally {
```

```
    setSubmitting(false);
  }
};

return (
  <div className="report-layout">
    <div className="bg-image"></div>

    <div className="content-wrapper">
      <div className="report-card">
        <h2>Report an Issue</h2>
        <p className="subtitle">
          Help improve village infrastructure by reporting issues
        </p>

        {error && <p style={{ color: "red", marginBottom: "1rem", fontWeight: "bold" }}>{error}</p>}
        {success && <p style={{ color: "green", marginBottom: "1rem", fontWeight: "bold" }}>{success}</p>}

        <form className="report-form" onSubmit={handleSubmit}>
          <div className="form-group">
            <label>Issue Title</label>
```

```
  <input
    name="title"
    value={formData.title}
    onChange={handleChange}
    placeholder="Short issue title"
    required
  />
</div>


<div className="form-group">
  <label>Category</label>
  <select
    name="category"
    value={formData.category}
    onChange={handleChange}
    required
  >
    <option value="">Select category</option>
    <option>Water</option>
    <option>Electricity</option>
    <option>Road</option>
    <option>Drainage</option>
```

```
    <option>Temple</option>
  </select>
</div>

<div className="form-group">
  <label>Description</label>
  <textarea
    name="description"
    rows="4"
    value={formData.description}
    onChange={handleChange}
    placeholder="Describe the issue"
    required
  />
</div>

<div className="form-group">
  <label>Upload Image</label>
  <input
    type="file"
    accept="image/*"
    onChange={handleFileChange}
```

```
            />
          </div>

          <div className="form-group">
            <label>Location</label>
            <input
              name="location"
              value={formData.location}
              onChange={handleChange}
              placeholder="Street / Ward / Landmark"
            />
          </div>

          <button type="submit" className="submit-btn">
            Submit Issue
          </button>
        </form>
      </div>
    </div>
  </div>
);
}
```

5.5 Coding Standards Followed

The development of the *Voice of Village: Fund Transparency & Issue Reporting System* follows standard coding practices to ensure code quality, readability, and maintainability. Adhering to these standards helps in developing a clean and well-structured application.

The project follows proper **naming conventions**, where variables, functions, and database tables are given meaningful and descriptive names. This improves code clarity and makes it easier to understand the functionality of different components.

The code is written in a **modular and structured manner**, separating concerns into different layers such as routes, controllers, middleware, and configuration files. This organization helps in maintaining the code and simplifies debugging and updates.

Consistent **indentation and formatting** are maintained throughout the codebase to improve readability. Proper spacing and alignment make the code easier to review and understand.

Code reusability is emphasized by creating reusable functions and components wherever possible. This reduces duplication and follows the DRY (Don't Repeat Yourself) principle.

Comments are added where necessary to explain complex logic, improving the understanding of the code for future developers or reviewers.

The project also follows **security best practices**, such as hashing passwords, validating user inputs, and protecting routes using authentication mechanisms. Version control is maintained using Git, allowing tracking of changes and ensuring proper management of the codebase.

Overall, these coding standards ensure that the system is clean, efficient, and easy to maintain, supporting future enhancements and scalability.

6. Testing

6.1 Testing Strategy

The *Voice of Village: Fund Transparency & Issue Reporting System* follows a systematic testing strategy to ensure that the application functions correctly, efficiently, and securely. Testing is carried out at different stages of development to identify and fix issues early.

The project adopts a **module-based testing approach**, where each module such as authentication, issue reporting, and comments is tested individually before integration. This helps in detecting errors at an early stage and ensures that each component works as expected.

Both **manual testing** and **API testing** are performed during development. Tools like Thunder Client are used to test backend APIs by sending requests and verifying responses. This ensures that all endpoints function correctly.

After individual module testing, **integration testing** is performed to verify that different modules work together without issues. This includes testing the interaction between frontend and backend, as well as database operations.

The system is also tested for **edge cases and error handling**, such as invalid inputs, unauthorized access, and missing data. Proper error messages and responses are verified to ensure a smooth user experience.

Additionally, **user interface testing** is conducted to ensure that the application is easy to use, responsive, and visually consistent across different screens.

Overall, the testing strategy ensures that the system is reliable, secure, and performs efficiently under different scenarios, providing a smooth experience for users.

6.2 Unit Testing

Unit testing is performed to verify the functionality of individual components of the *Voice of Village: Fund Transparency & Issue Reporting System*. Each module is tested independently to ensure that it works correctly before integration with other parts of the system.

In this project, unit testing mainly focuses on backend functions such as user authentication, issue submission, data retrieval, and status updates. Each API endpoint is tested separately to check whether it produces the expected output for given inputs.

The authentication module is tested to ensure proper registration, login, password validation, and token generation. The issue reporting module is tested to verify that issues are correctly stored in the database with all required details. Similarly, the comment module is tested to ensure that comments are added and retrieved accurately.

Different test cases are used, including valid inputs, invalid inputs, and boundary conditions, to ensure robustness. Error handling is also verified to confirm that appropriate responses are returned for incorrect or incomplete data.

Unit testing helps in identifying bugs at an early stage and ensures that each component of the system functions correctly in isolation, improving overall system reliability.

6.3 Integration Testing

Integration testing is performed to ensure that different modules of the *Voice of Village: Fund Transparency & Issue Reporting System* work together correctly as a unified system. After unit testing individual components, they are integrated and tested for proper data flow and interaction.

In this project, integration testing mainly focuses on the interaction between the frontend, backend, and database. The complete workflow—from user action in the frontend to data processing in the backend and storage in the database—is verified.

The key integration testings performed are as follows:

- **Authentication Flow Integration:** Tested complete login flow including token generation, token validation, and access to protected routes.
- **Issue Reporting Flow:** Verified that issues submitted from the frontend are correctly saved in the database and can be retrieved through APIs.
- **Issue Status Update Flow:** Checked whether authorized users can update issue status and the changes are reflected correctly in the system.

- **Comment System Integration:** Ensured that comments added by users are stored and displayed correctly for each issue.
- **Role-Based Access Control**: Verified that users can only access functionalities based on their roles and unauthorized actions are restricted.
- **Error Handling and Response Flow:** Tested how the system handles invalid requests, missing data, and unauthorized access across modules.

Overall, integration testing ensures that all modules interact smoothly and the system functions as a complete and reliable application.

6.4 System Testing

System testing is performed to evaluate the complete *Voice of Village: Fund Transparency & Issue Reporting System* as a whole and ensure that it meets the specified requirements. In this phase, the entire application is tested in an integrated environment to verify overall functionality, performance, and reliability.

The system is tested by simulating real-world scenarios where different users interact with the application. This includes activities such as user registration, login, issue submission, issue tracking, status updates by authorities, and commenting. The goal is to ensure that all features work together correctly and provide the expected output.

Both **functional testing** and **non-functional testing** are carried out during this phase. Functional testing ensures that all system features operate as intended, while non-functional testing focuses on aspects such as performance, usability, and security.

The key system testings performed include:

- **End-to-End Testing**: Verified the complete workflow from user login to issue resolution, ensuring smooth execution of all processes.
- **User Role Testing**: Ensured that different users (villager, Sarpanch, admin, government official) can access only their respective functionalities.
- **Performance Testing**: Checked system response time and behavior under multiple requests to ensure efficient performance.

- **Usability Testing**: Verified that the interface is user-friendly, easy to navigate, and suitable for users with minimal technical knowledge.
- **Security Testing**: Tested authentication, authorization, and data protection mechanisms to ensure secure system operation.
- **Error Handling Testing**: Ensured that the system handles errors gracefully and provides appropriate messages to users.

Overall, system testing confirms that the application is stable, reliable, and ready for deployment, providing a seamless experience to all users.

6.5 Test Cases and Results

Multiple test cases were designed and executed to validate the core functionalities of the EcoTrace system under different scenarios. The testing covered authentication, product and batch management, AI evaluation, transport tracking, lab verification, and QR-based access.

Test Case	Input Scenario	Expected Result	Actual Result	Status
User Registration	Valid email and password	Account created successfully	Account created	Passed
User Registration	Duplicate email	Error message shown	Error displayed	Passed
User Login	Correct credentials	Login successful, token generated	Login successful	Passed
User Login	Incorrect password	Access denied	Error displayed	Passed
Submit Issue	Valid product data	Product created and stored	Product created	Passed
Get All Issues	Valid materials and percentages	Batch created successfully	Batch created	Passed
Update Issue Status	No change in materials	Auto verification	Verified automatically	Passed
Add Comment	Minor change in composition	AI scoring triggered	Score generated	Passed
Unauthorized Access	Major change	Lab verification required	Marked for lab	Passed

Role-Based Access	Valid batch data	Score and insights generated	Score displayed	Passed
Error Handling	Valid report data	Report submitted	Report created	Passed
Admin Approval	Approve lab report	Batch verified	Verified	Passed
Admin Rejection	Reject lab report	Batch rejected	Rejected	Passed

Table 6.5.1 Test Cases and Results

6.6 Bug Reporting and Tracking

Bug reporting and tracking is an important part of the development process to ensure the reliability and quality of the *Voice of Village: Fund Transparency & Issue Reporting System*. During testing, any issues or defects identified in the system are recorded, analyzed, and resolved systematically.

Bugs are identified during unit testing, integration testing, and system testing phases. Each bug is documented with details such as description, module affected, steps to reproduce, and expected versus actual results. This helps in understanding the issue clearly and fixing it efficiently.

Common types of bugs encountered include validation errors, incorrect API responses, authentication issues, and data handling problems. These issues are tracked and resolved during development to ensure smooth system functionality.

Version control tools like Git are used to manage changes and track bug fixes. Each fix is updated in the codebase with proper commits, making it easier to maintain a history of changes.

After fixing a bug, the system is retested to ensure that the issue is resolved and does not affect other functionalities. This process helps in maintaining system stability and improving overall performance.

Overall, effective bug reporting and tracking ensures that the system is robust, reliable, and free from critical errors before deployment.

6.7 Quality Assurance Standards

Quality assurance is an essential part of the *Voice of Village: Fund Transparency & Issue Reporting System* to ensure that the application meets required standards of performance, reliability, and usability

The project follows a **systematic testing approach**, including unit testing, integration testing, and system testing, to verify that all functionalities work correctly. Each module is tested individually and then as part of the complete system to ensure consistency.

Code quality standards are maintained by following proper coding conventions, meaningful naming, and modular design. This improves readability, maintainability, and reduces the chances of errors.

Security standards are followed by implementing authentication and authorization mechanisms using JWT. Passwords are securely hashed, and access to protected routes is restricted based on user roles. **Usability standards** are maintained by designing a simple and intuitive user interface. The application is tested to ensure smooth navigation and ease of use for all types of users. Regular **bug tracking and resolution** are carried out to identify and fix issues during development. Retesting is performed after fixing bugs to ensure system stability.

Overall, these quality assurance practices ensure that the system is reliable, secure, user-friendly, and ready for deployment.

7. Results and Discussion

7.1 Output Screenshots

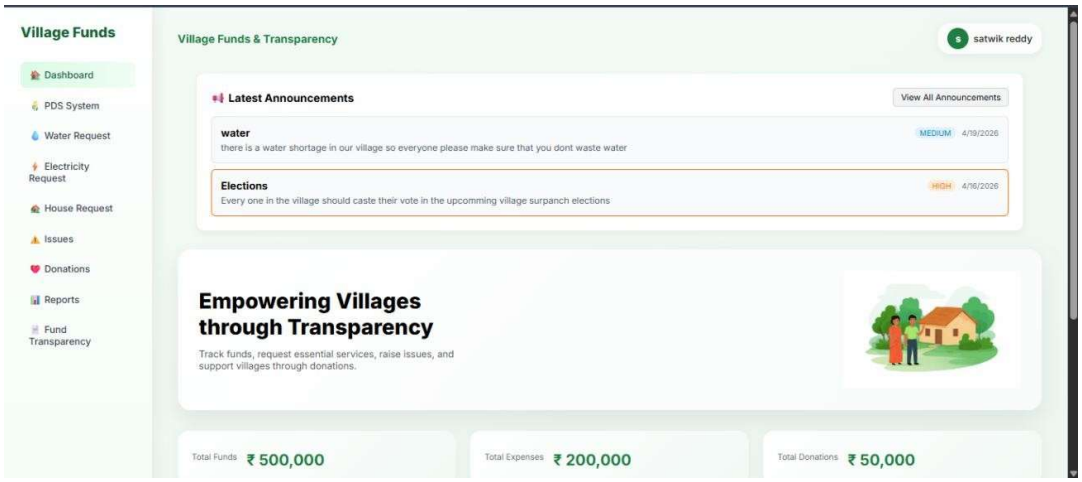

Fig 7.1.1 Villager

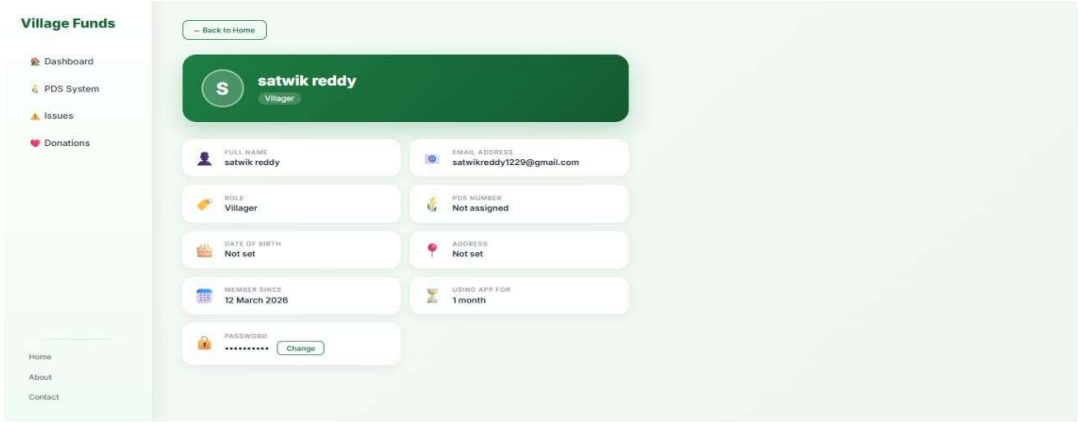

Fig 7.1.2 villager profile

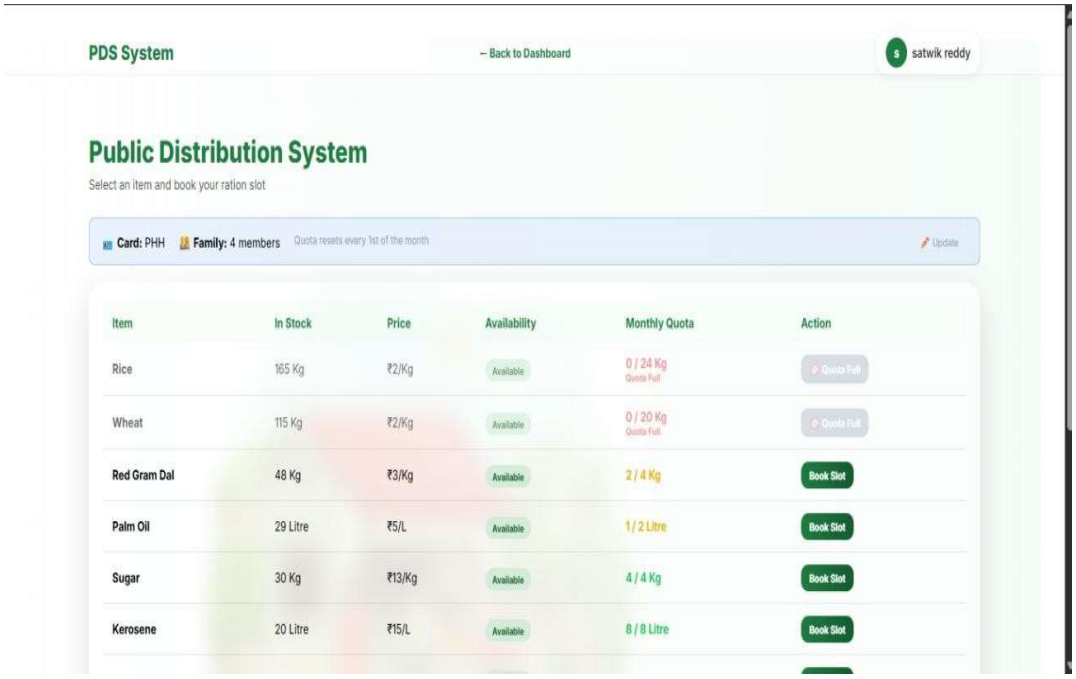

FIG 7.1.3.1 PDS System

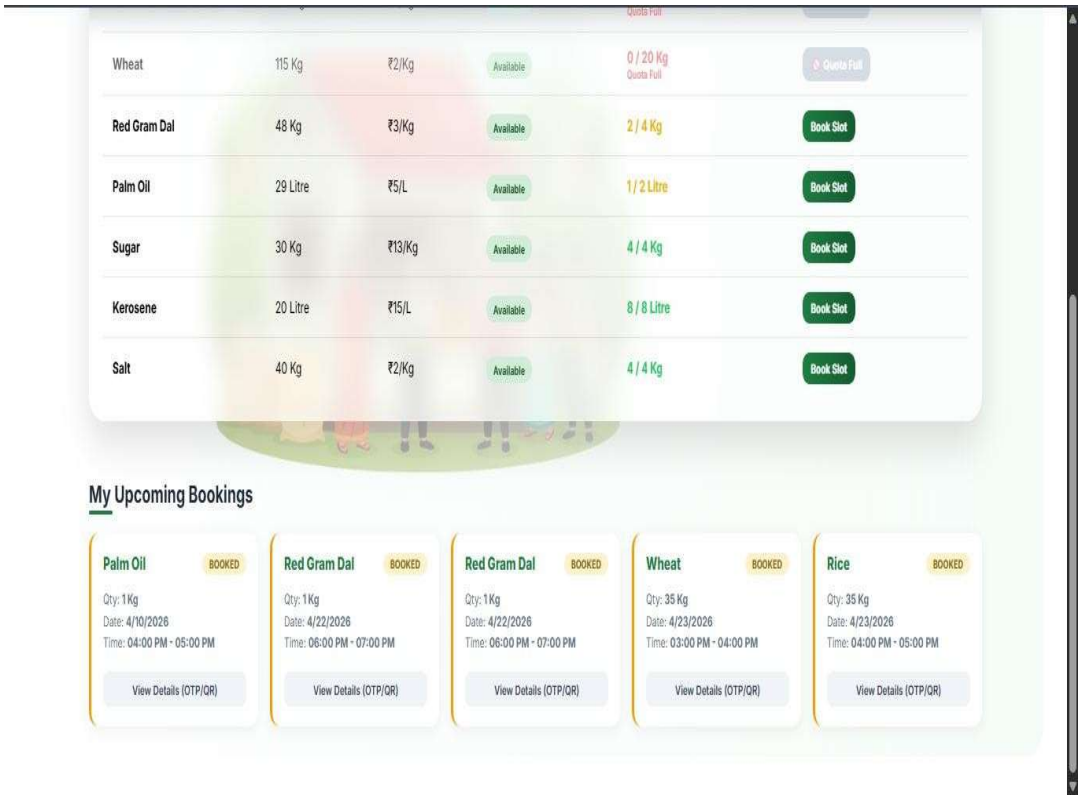

Fig 7.1.3.2 PDS System

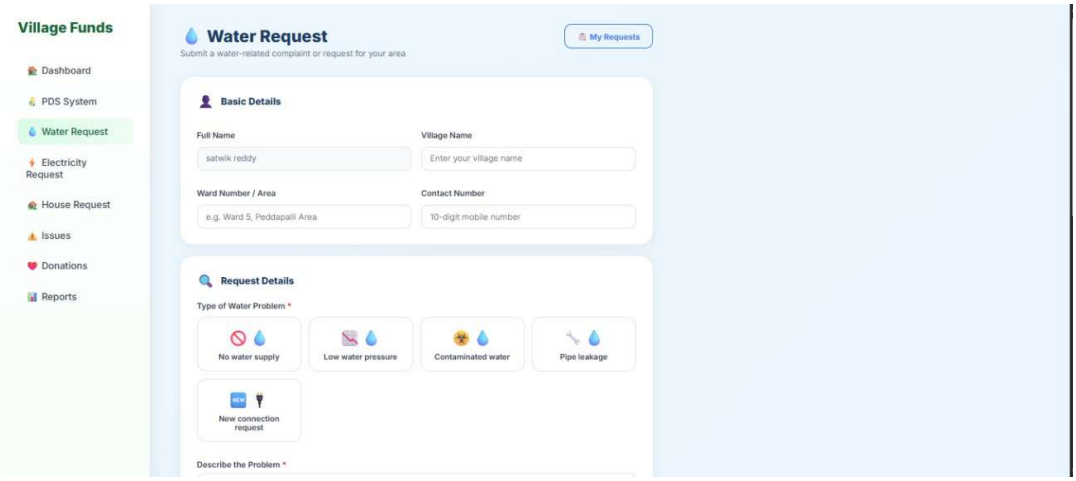

Fig 7.1.4.1 water request

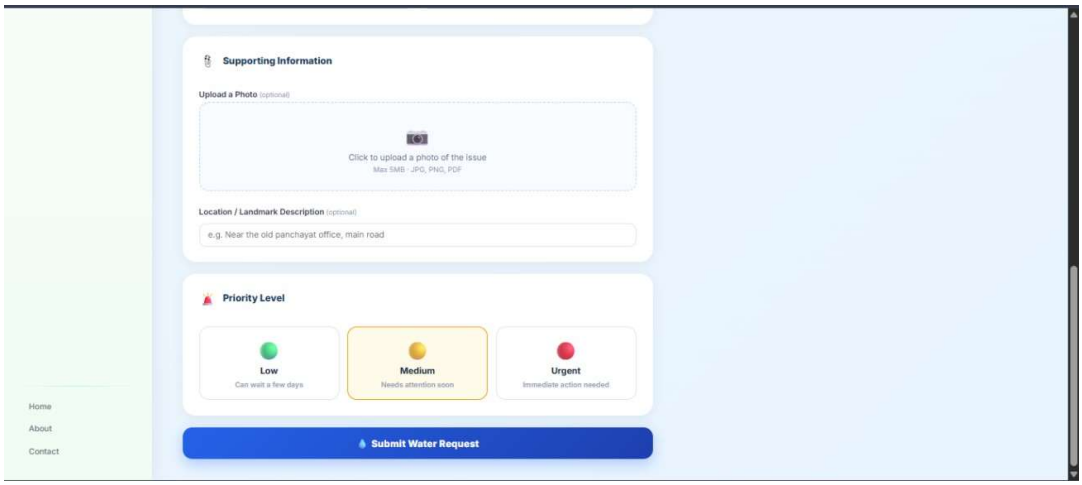

Fig 7.1.4.2 water request

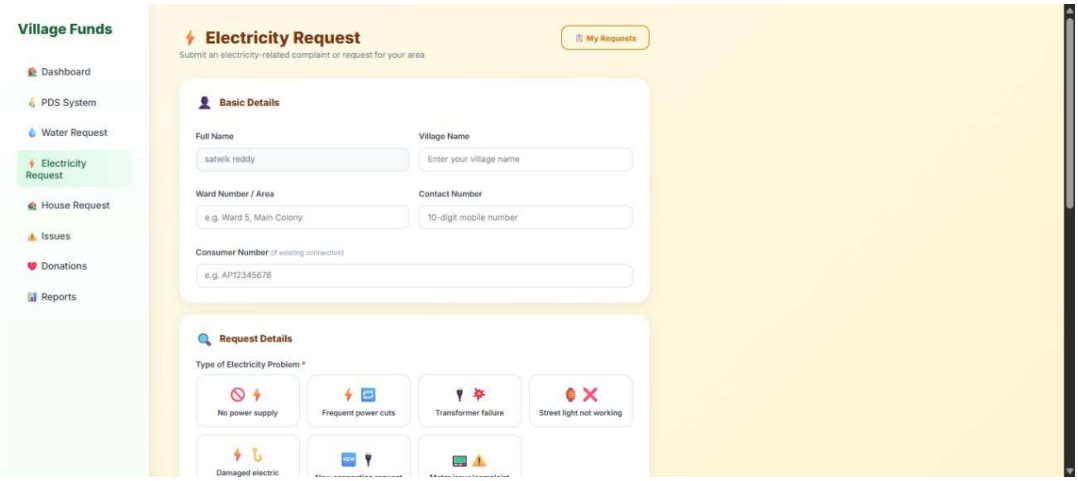

Fig 7.1.5.1 Electricity request

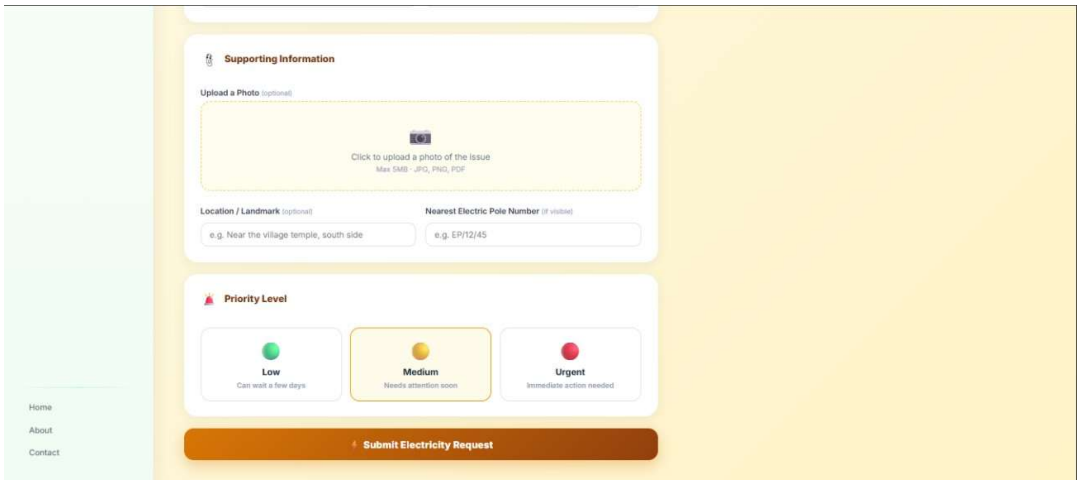

Fig 7.1.5.2 Electricity request

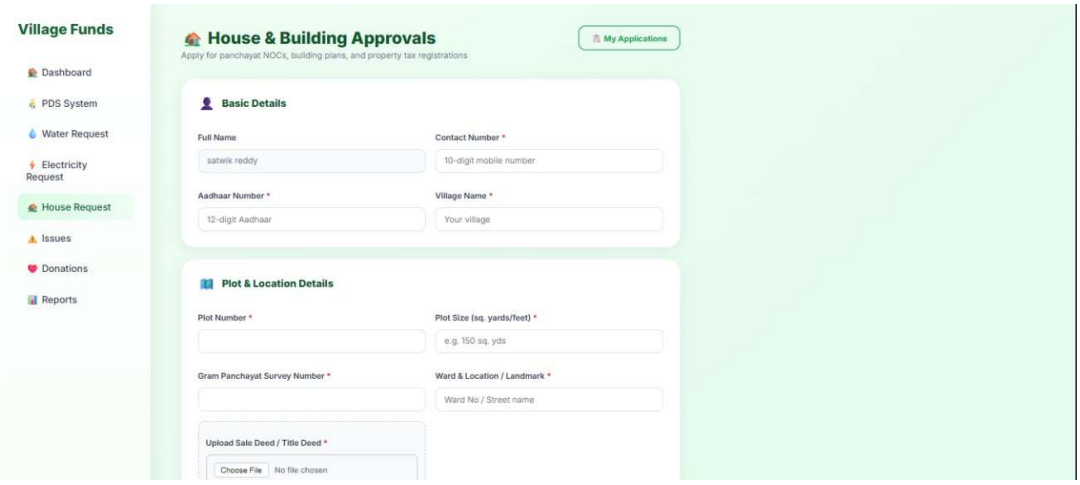

Fig 7.1.6.1 House request

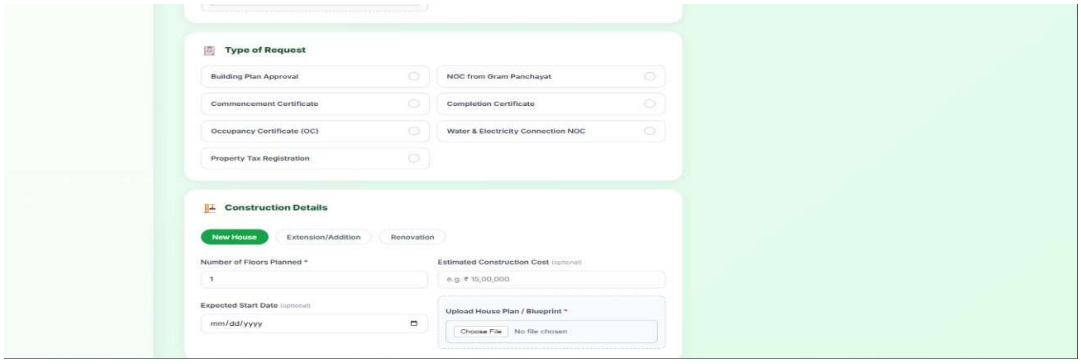

Fig 7.1.6.2 House Request

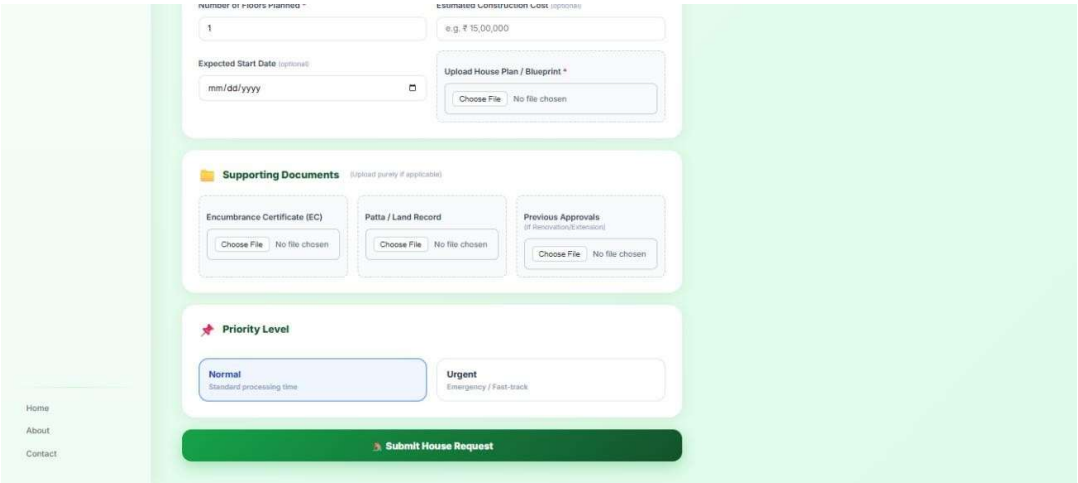

Fig 7.1.6.3 House request

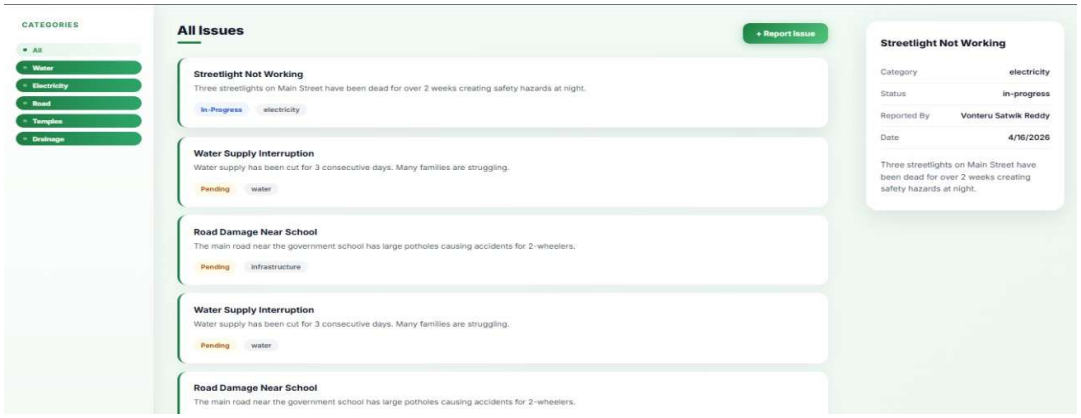

Fig 7.1.7 Issues Reporting

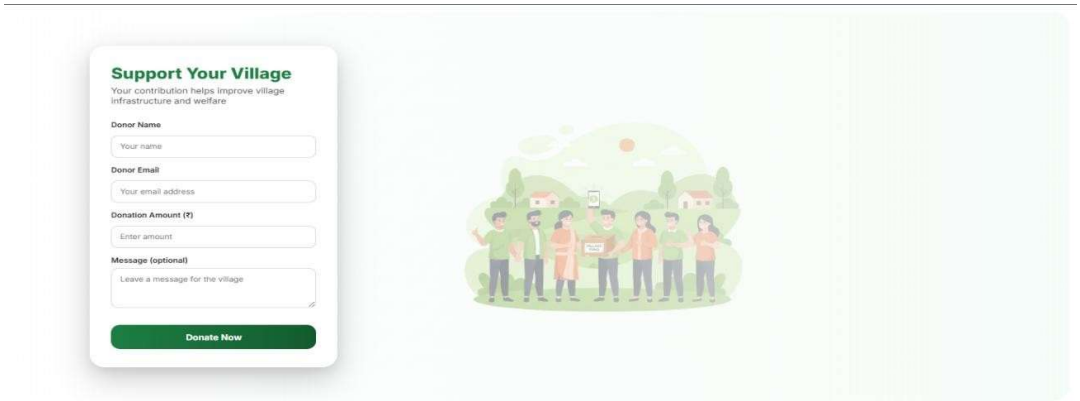

Fig 7.1.8 Donations page

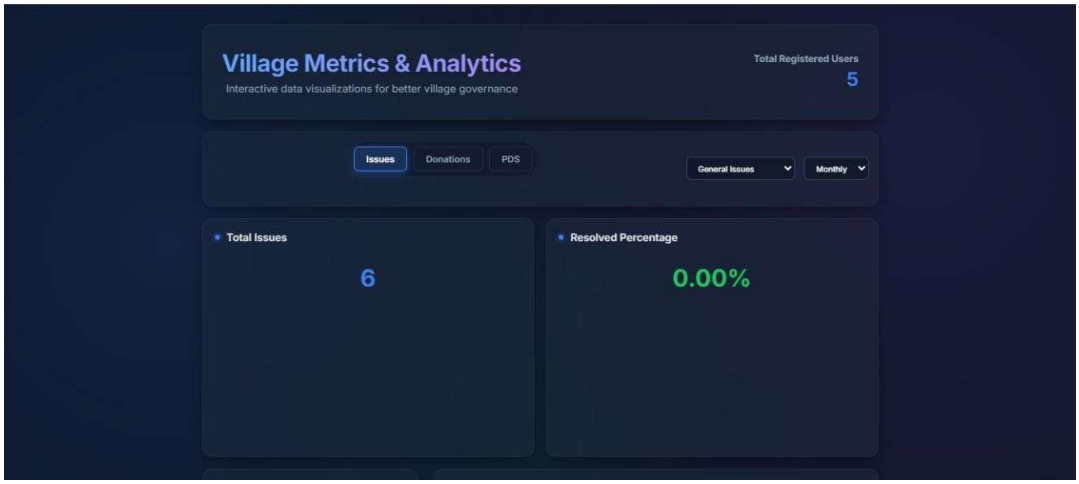

Fig 7.1.9 Reports

Fig 7.1.8.2 Reports

7.2 Results Interpretation

The results of the *Voice of Village: Fund Transparency & Issue Reporting System* demonstrate that the application successfully achieves its objective of improving transparency and communication in village governance.

The system enables users to report issues efficiently and track their progress in real time. This ensures that citizens are aware of the status of their complaints, which increases trust and engagement. The issue management functionality allows authorities to update and resolve issues in a structured manner, reducing delays and miscommunication.

The implementation of role-based access control ensures that different users can perform only their designated tasks, improving system security and organization. The comment feature enhances interaction between citizens and authorities, making the process more transparent and collaborative.

The integration between frontend, backend, and database works smoothly, ensuring accurate data flow and reliable system performance. The testing results confirm that all major functionalities operate correctly under different scenarios.

Overall, the system provides an effective solution for digitizing village-level governance, improving efficiency, accountability, and user participation.

7.3 Performance Evaluation

The performance of the *Voice of Village: Fund Transparency & Issue Reporting System* is evaluated based on factors such as response time, system reliability, and efficiency of operations. The system demonstrates satisfactory performance under normal usage conditions.

The application provides **fast response times** for most operations such as user login, issue submission, and data retrieval. The use of efficient backend processing and structured database queries ensures that requests are handled quickly.

The system shows good **reliability**, as it performs consistently without major failures during testing. Error handling mechanisms ensure that unexpected inputs or issues do not cause system crashes.

In terms of **efficiency**, the system is capable of handling multiple user requests simultaneously without significant delays. The modular design and optimized API communication contribute to smooth performance.

The **frontend interface** is responsive and provides a seamless user experience across different devices. Pages load quickly, and navigation between features is smooth.

Overall, the system meets the expected performance requirements and provides a stable and efficient platform for users. Further optimization can be done in the future to support larger-scale deployment and increased user load.

7.4 Comparative Results

The performance and effectiveness of the *Voice of Village: Fund Transparency & Issue Reporting System* were evaluated by comparing it with existing approaches used in village-level governance.

In traditional systems, issue reporting is mostly manual, leading to delays, lack of tracking, and poor communication. In contrast, the proposed system provides a **digital and structured platform** where issues can be reported and tracked in real time, significantly improving efficiency.

Existing systems often lack transparency in processes and fund utilization. The proposed system enhances **transparency and accountability** by allowing users to monitor issue

status and view relevant updates, thereby increasing trust between citizens and authorities.
Communication in traditional methods is limited and unstructured, whereas the system introduces a **comment-based interaction mechanism**, enabling better coordination and clarity among stakeholders.

Additionally, the use of **role-based access control** ensures proper management of responsibilities, which is generally absent in existing approaches. This leads to better organization and reduced chances of misuse.
Overall, the proposed system outperforms existing methods by providing improved efficiency, transparency, communication, and user engagement, making it a more effective solution for village governance.

8. Conclusion and Future Scope

8.1 Summary of Work Done

The *Voice of Village: Fund Transparency & Issue Reporting System* was developed to address the challenges of transparency, accountability, and communication in village-level governance. The project focuses on providing a digital platform that enables citizens to report issues and track their resolution while also promoting transparency in administrative processes.

The system was designed and implemented using a full-stack approach. The frontend was developed using React.js to provide an interactive and user-friendly interface, while the backend was built using Node.js and Express.js to handle business logic and API communication. MySQL was used as the database for storing and managing structured data such as users, issues, and comments.

The project was developed in multiple phases, starting with requirement analysis and system design, followed by implementation of core modules such as user authentication, issue reporting, issue management, and commenting. Role-based access control was implemented to support different types of users including villagers, Sarpanch, administrators, and government officials.

Various testing techniques, including unit testing, integration testing, and system testing, were performed to ensure that the application functions correctly and efficiently. Bugs identified during testing were resolved, and quality assurance practices were followed to maintain system reliability and performance.

The final system successfully demonstrates a working solution for digital governance at the village level. It provides a centralized platform for issue reporting, real-time tracking, and improved communication between citizens and authorities.

Overall, the project achieves its objective of creating a transparent, efficient, and user-friendly system that enhances participation and accountability in village development.

8.2 Limitations

While the *Voice of Village: Fund Transparency & Issue Reporting System* provides an effective solution for improving transparency and issue management, it has certain limitations.

The system is developed for a **controlled environment** and is not integrated with real-time government databases or external administrative systems. This limits its ability to reflect actual government-level data and operations.

The **fund transparency module is conceptual and basic**, and does not include advanced features such as detailed financial analytics, auditing mechanisms, or real-time fund tracking.

The application currently depends on **manual updates by authorities** for issue status, which may lead to delays if updates are not performed regularly.

The system requires **internet connectivity**, which may be a challenge in rural areas with limited or unstable network access.

Advanced features such as **real-time notifications, mobile application support, and geolocation-based issue tracking** are not included in the current implementation.

The system is designed for **small to medium-scale usage** and may require further optimization and infrastructure support for large-scale deployment.

Despite these limitations, the system provides a strong foundation and can be enhanced in the future to overcome these challenges.

8.3 Challenges Faced

During the development of the *Voice of Village: Fund Transparency & Issue Reporting System*, several challenges were encountered at different stages of the project. These challenges helped in gaining practical experience and improving problem-solving skills.

One of the major challenges was **understanding and designing the system architecture**, especially in integrating frontend, backend, and database components. Ensuring smooth communication between these layers required careful planning and implementation.

Another challenge was **implementing authentication and role-based access control**. Managing different user roles such as villager, Sarpanch, administrator, and government official while ensuring secure access to specific functionalities required proper logic and testing.

Database design and management also posed challenges, particularly in establishing relationships between tables and maintaining data consistency. Writing efficient queries and handling data operations correctly required attention to detail.

During development, **API integration between frontend and backend** was a key challenge. Handling API requests, responses, and error cases while ensuring proper data flow required debugging and testing.

The team also faced issues related to **debugging and error handling**, especially when dealing with unexpected inputs or server-side errors. Identifying and resolving bugs efficiently was an important part of the development process.

Another challenge was **time management and coordination**, as the project involved multiple modules that needed to be completed within a limited timeframe.

Additionally, ensuring a **user-friendly interface** that is simple and accessible for all users required multiple iterations and improvements.

Despite these challenges, they were successfully addressed through continuous learning, testing, and teamwork, leading to the successful completion of the project.

8.4 Future Enhancements

- The *Voice of Village: Fund Transparency & Issue Reporting System* provides a strong foundation for improving transparency and communication in village governance. However, there are several enhancements that can be implemented in the future to extend its functionality and effectiveness.
- One of the major enhancements is the development of a **mobile application**, which would make the system more accessible to users, especially in rural areas where mobile usage is more common than desktops.
- The system can be improved by adding **real-time notifications**, such as SMS or push notifications, to inform users about issue updates, status changes, and important announcements.
- Integration with **government databases and e-governance systems** can be implemented to provide real-time and accurate data related to fund allocation and administrative actions.
- The **fund transparency module** can be enhanced with advanced features such as detailed financial reports, visual analytics, and auditing mechanisms to provide deeper insights into fund utilization.
- Incorporating **geolocation features** can help users report issues with location data, making it easier for authorities to identify and resolve problems efficiently.
- The system can also include **AI-based analytics** to prioritize issues based on severity, frequency, or impact, helping authorities make better decisions.
- Improving **security features**, such as multi-factor authentication and advanced data protection mechanisms, can further enhance system safety.
- Additionally, the system can be scaled for **larger deployments**, supporting multiple villages or regions with centralized monitoring and management.
- Overall, these enhancements can transform the system into a more powerful, scalable, and real-world-ready solution for digital governance

9. References

These sources include official documentation, tutorials, and technical resources that helped in understanding and implementing various technologies.

Technical Publications

- Pressman, R. S., *Software Engineering: A Practitioner's Approach*
- Sommerville, I., *Software Engineering*

Websites and Documentation

- Node.js Official Documentation – https://nodejs.org
- Express.js Documentation – https://expressjs.com
- React Documentation – https://react.dev
- MySQL Documentation – https://www.mysql.com
- JWT Introduction – https://jwt.io
- Railway databse- https://railway.com/

Tools and Platforms

- Git – https://git-scm.com
- GitHub – https://github.com
- Postman / Thunder Client for API testing

Other Sources

- Online tutorials and developer forums such as Stack Overflow
- YouTube tutorials related to full-stack web development

These references provided guidance for designing, developing, and testing the system effective

10. Appendices

A. SDLC Forms

This section includes the Software Development Life Cycle (SDLC) related documents used during the development of the *Voice of Village: Fund Transparency & Issue Reporting System*. These forms help in documenting requirements, design decisions, and testing activities in a structured manner.

1. Software Requirements Specification (SRS): The SRS document defines the overall requirements of the system, including functional and non-functional requirements, user roles, and system constraints. It serves as a blueprint for system design and development.

2. Feasibility Study Report: This document analyzes the technical, economic, and operational feasibility of the project. It ensures that the system is practical, cost-effective, and suitable for implementation.

3. Design Documents: These include system architecture diagrams, database design (ER diagram), and UML diagrams such as use case, class, sequence, and activity diagrams. These documents provide a clear understanding of the system structure and workflow.

4. Test Reports: Test reports contain details of test cases, test results, and bug tracking. They help in verifying that the system functions correctly and meets the specified requirements.

5. Requirement Analysis Documents: These documents capture the initial requirements gathered from the problem statement and user needs. They guide the overall development process.

All these SDLC forms ensure that the project follows a structured development process and maintains proper documentation throughout its lifecycle.

B. Gantt Chart

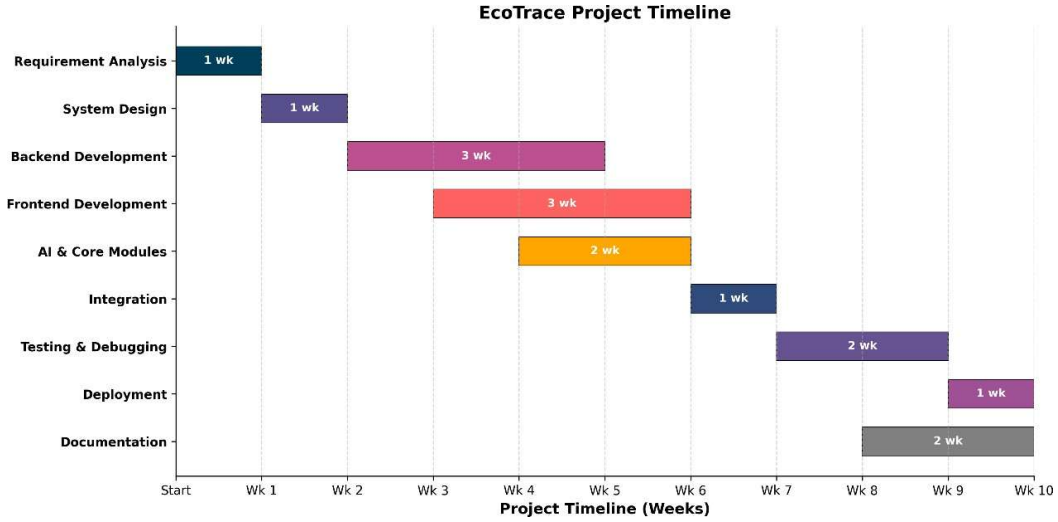

Fig 10.2 Gantt Chart

C. Ethical Considerations and Consent

The development of the *Voice of Village: Fund Transparency & Issue Reporting System* considers important ethical aspects to ensure responsible and fair usage of the platform. The system is designed to promote transparency, accountability, and user trust while protecting user data and privacy.

One of the key considerations is data privacy and confidentiality. Personal information such as user details and reported issues is handled securely and is not shared with unauthorized parties. Appropriate measures such as authentication and access control are implemented to protect sensitive data.

The system ensures fair and unbiased usage, where all users have equal access to report issues and view information. No discrimination is made based on user role beyond functional permissions required for system operation.

User consent is considered during registration, where users agree to provide their information for system usage. Users are informed about how their data will be used within the platform.

The platform also promotes transparency and accountability, ensuring that actions such as issue updates and comments are visible and traceable. This reduces the chances of misuse and encourages responsible behavior among users and authorities.

Additionally, the system is designed to avoid misuse by implementing restrictions on unauthorized access and improper activities.

Overall, these ethical considerations ensure that the system operates in a responsible, secure, and user-centric manner while maintaining trust among all stakeholders.

D. Plagiarism Report

E. Source Code Repository and Deployment Links

The source code repository is maintained using GitHub, where all project files, including frontend, backend, and database-related configurations, are stored. Version control is used to track changes, manage updates, and maintain the project history.

The deployment links provide access to the live application (if deployed), allowing users to interact with the system in a real-time environment. These links include both frontend and backend endpoints.

GitHub Repository Link: https://github.com/Satwik1229/Voice-Of-Village

F. Soft Copy / Drive Link for all project related files

Google Drive Link:
https://drive.google.com/drive/folders/1PhPBj0fZxbn6AwU2r0nR3BNxFDfQjhKW

Deployment Link: https://voice-village-host--udayyadav200323.replit.app

Overall, the project provides a technological approach to achieving sustainable development by improving governance, ensuring transparency in fund utilization, and strengthening communication between citizens and authorities.

G. Journal / Conference paper published on project

IRJMETS

International Research Journal Of Modernization in Engineering Technology and Science

(Peer-Reviewed, Open Access, Fully Refereed International Journal)

e-ISSN: 2582-5208

Ref: IRJMETS/Certificate/Volume 08/Issue 04/80400095379

DOI: https://www.doi.org/10.56726/IRJMET593756

Date: 07/04/2026

Certificate of Publication

This is to certify that author "**Uday Peraboina**" with paper ID "**IRJMETS80400095379**" has published a paper entitled "**VOICE OF VILLAGE: FUND TRANSPARENCY AND ISSUE REPORTING SYSTEM**" in International Research Journal of Modernization in Engineering Technology and Science (IRJMETS), Volume 08, Issue 04, April 2026

Editor in Chief

IRJMETS Impact Factor 8.187

We Wish For Your Better Future

www.irjmets.com

Google ISSUU Academia.edu doi Crossref

IRJMETS

International Research Journal Of Modernization in Engineering Technology and Science

(Peer-Reviewed, Open Access, Fully Refereed International Journal)

e-ISSN: 2582-5208

Ref: IRJMETS/Certificate/Volume 08/Issue 04/80400095379

DOI: https://www.doi.org/10.56726/IRJMETS93756

Date: 07/04/2026

Certificate of Publication

This is to certify that author "**V. Satwik Reddy**" with paper ID "***IRJMETS80400095379***" has published a paper entitled "***VOICE OF VILLAGE: FUND TRANSPARENCY AND ISSUE REPORTING SYSTEM***" in International Research Journal of Modernization in Engineering Technology and Science (IRJMETS), Volume 08, Issue 04, April 2026

Editor in Chief

IRJMETS Impact Factor 8.187

We Wish For Your Better Future

www.irjmets.com

Google ISSUU Academia.edu doi Crossref

International Research Journal Of Modernization in Engineering Technology and Science

(Peer-Reviewed, Open Access, Fully Refereed International Journal)

e-ISSN: 2582-5208

Ref: IRJMETS/Certificate/Volume 08/Issue 04/80400095379

DOI: https://www.doi.org/10.56726/IRJMETS93756

Date: 07/04/2026

Certificate of Publication

This is to certify that author "**M. Manikanth Reddy**" with paper ID "**IRJMETS80400095379**" has published a paper entitled "**VOICE OF VILLAGE: FUND TRANSPARENCY AND ISSUE REPORTING SYSTEM**" in International Research Journal of Modernization in Engineering Technology and Science (IRJMETS), Volume 08, Issue 04, April 2026

Editor in Chief

IRJMETS Impact Factor 8.187

We Wish For Your Better Future

www.irjmets.com